



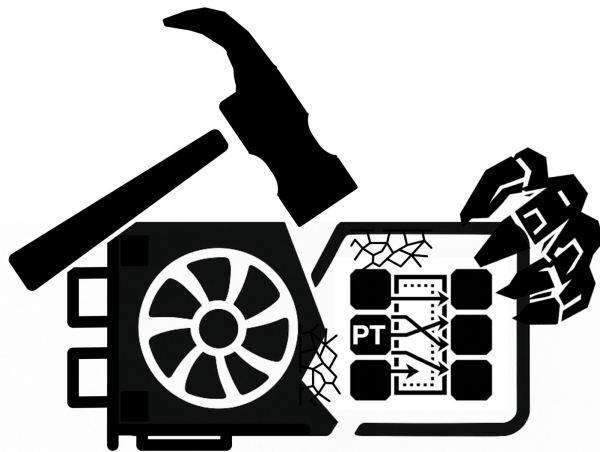
GPUBreach: Privilege Escalation Attacks on GPUs using Rowhammer

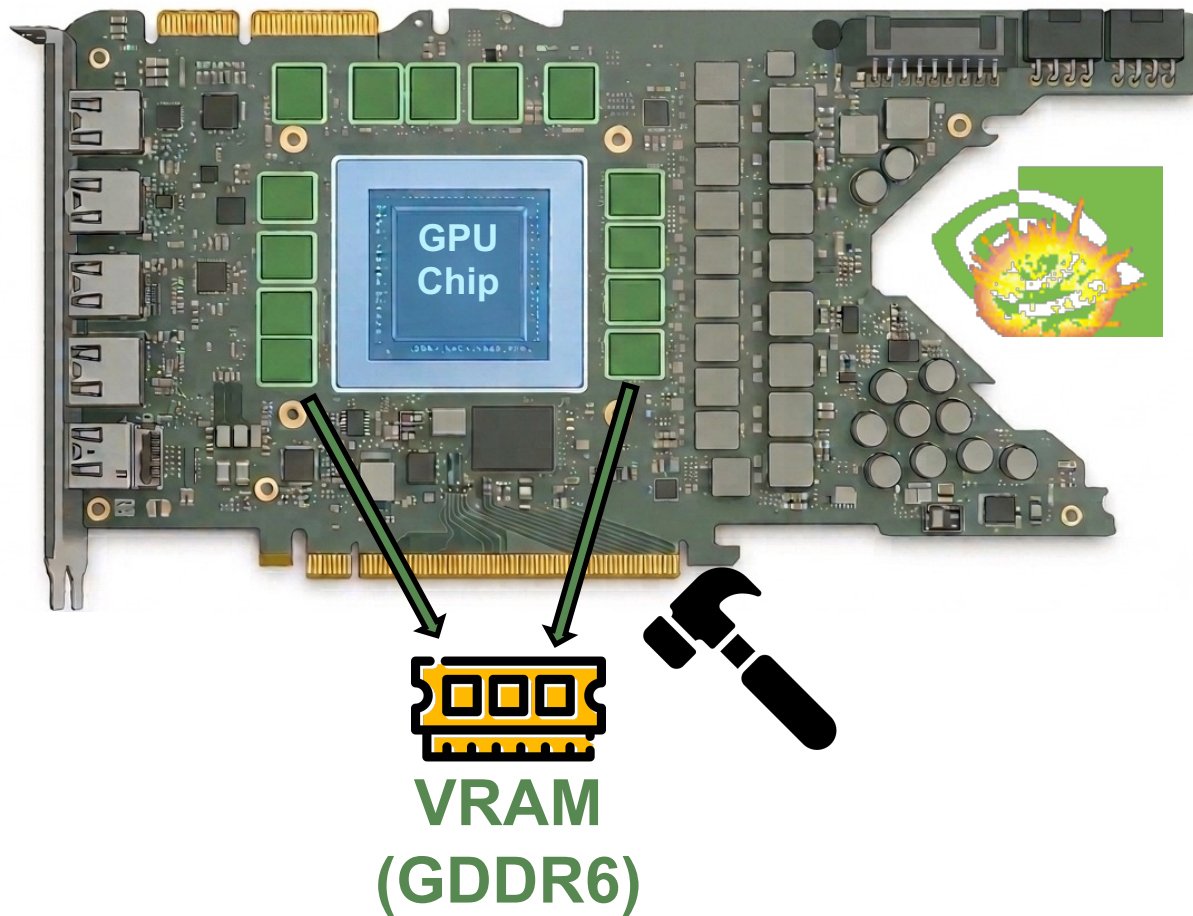
Presenters:

Chris S. Lin
Yuqin Yan
Guozhen Ding

In collaboration with:

Joyce Qu, Joseph Zhu, David Lie, and Gururaj Saileshwar



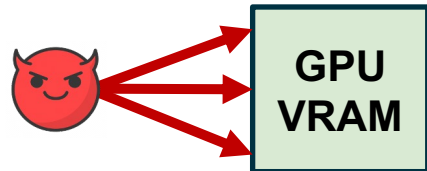


Executive Summary of GPUBreach

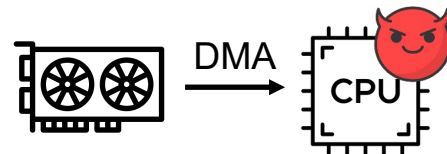
Flip bits on GPU Memory to Corrupt Page Tables



Enables GPU Privilege Escalation Attacks



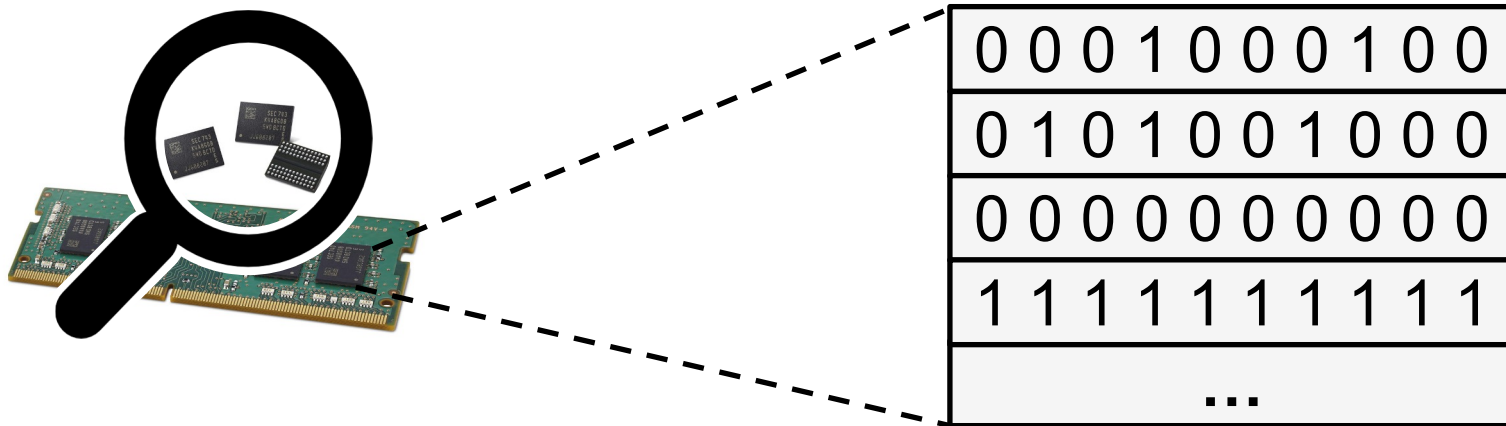
System-Wide Root Privileges on Host





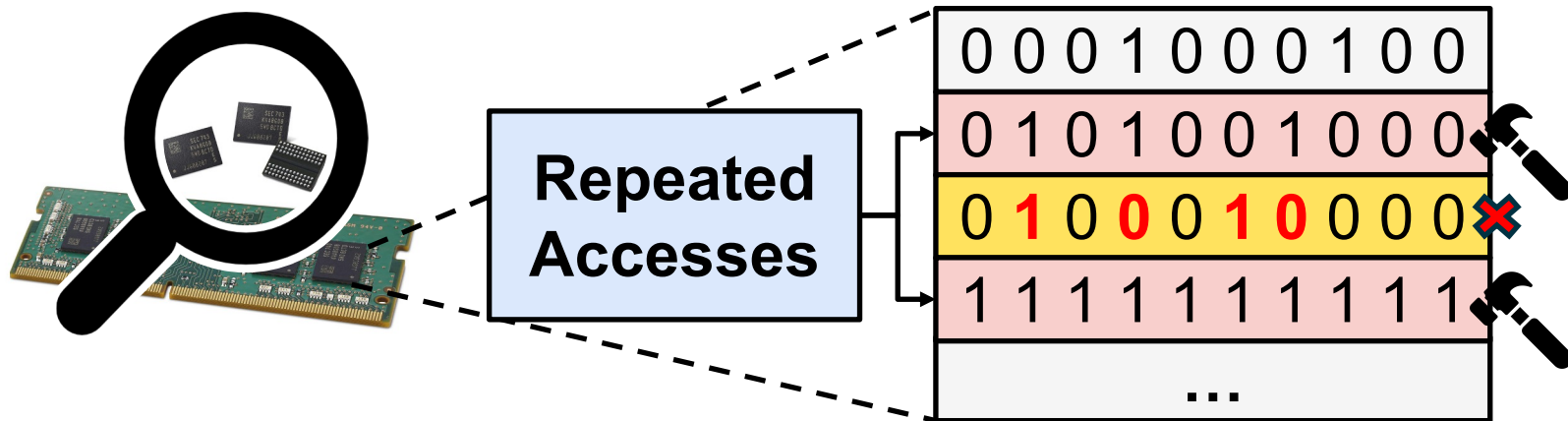
What's Rowhammer (on GPUs)?

The Rowhammer Vulnerability



Memories are organized into **rows** of electrical cells
(binary data)

The Rowhammer Vulnerability



Rapid Accesses to Rows Can Flip Bits in its Neighbour!

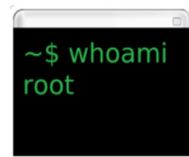
CPU Rowhammer Exploits

Rowhammer Exploits are well-studied on CPUs



Mark Seaborn
(Blackhat 2015)

Kernel Exploit:
Page Table Tampering



Root!

NaCl Exploit:
Corrupt Instruction

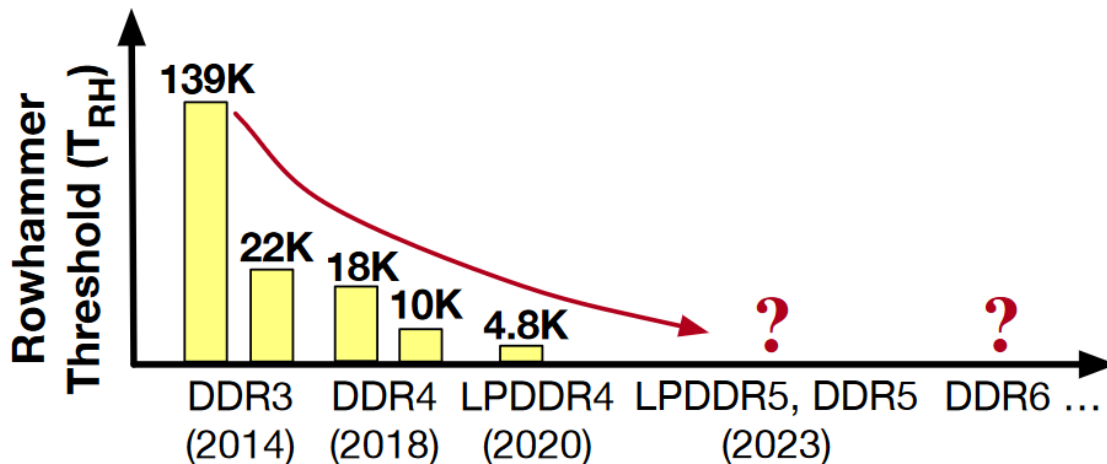
`jmp rax`



`jmp rcx`

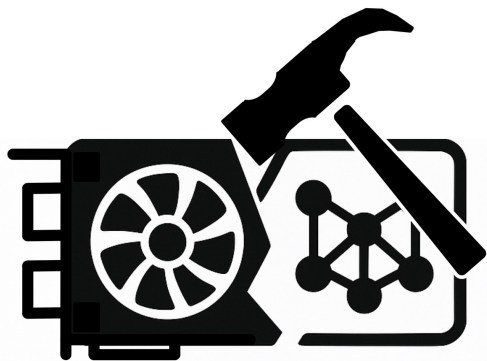
And many more enabled by bit flips...

State of Rowhammer



Still Not Fixed! And Getting Worse!

Rowhammer on GPUs



GPUHammer [SEC'25]

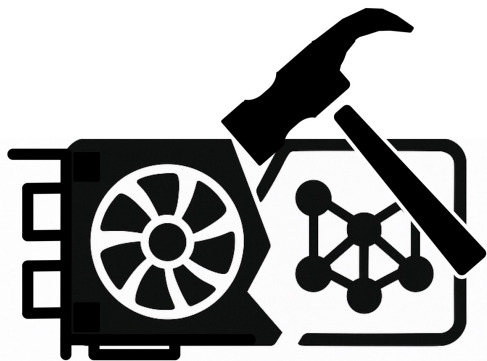
Found bit flips on RTX A6000
GPU, GDDR6 Memory

Chris S. Lin, Joyce Qu
Gururaj Saileshwar



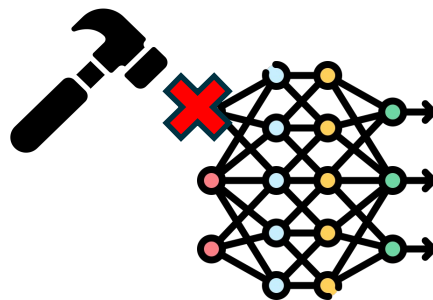
<https://www.utoronto.ca/news/how-three-u-t-researchers-discovered-gpu-vulnerability-threatened-ai-models>

Rowhammer on GPUs



GPUHammer [SEC'25]

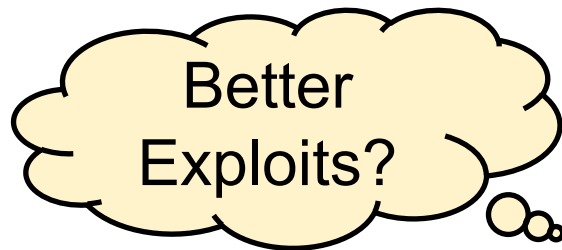
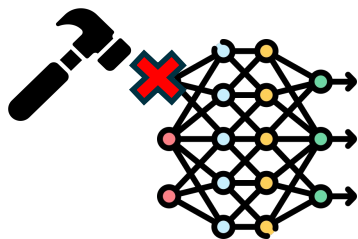
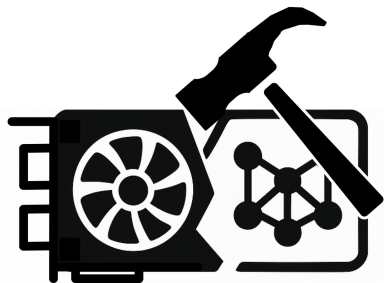
Found bit flips on RTX A6000
GPU, GDDR6 Memory



Accuracy 0%

Rowhammer on GPUs

But GPU Rowhammer Has A Huge Gap...



What about when:

No time-slicing while sharing?

No sharing at all?

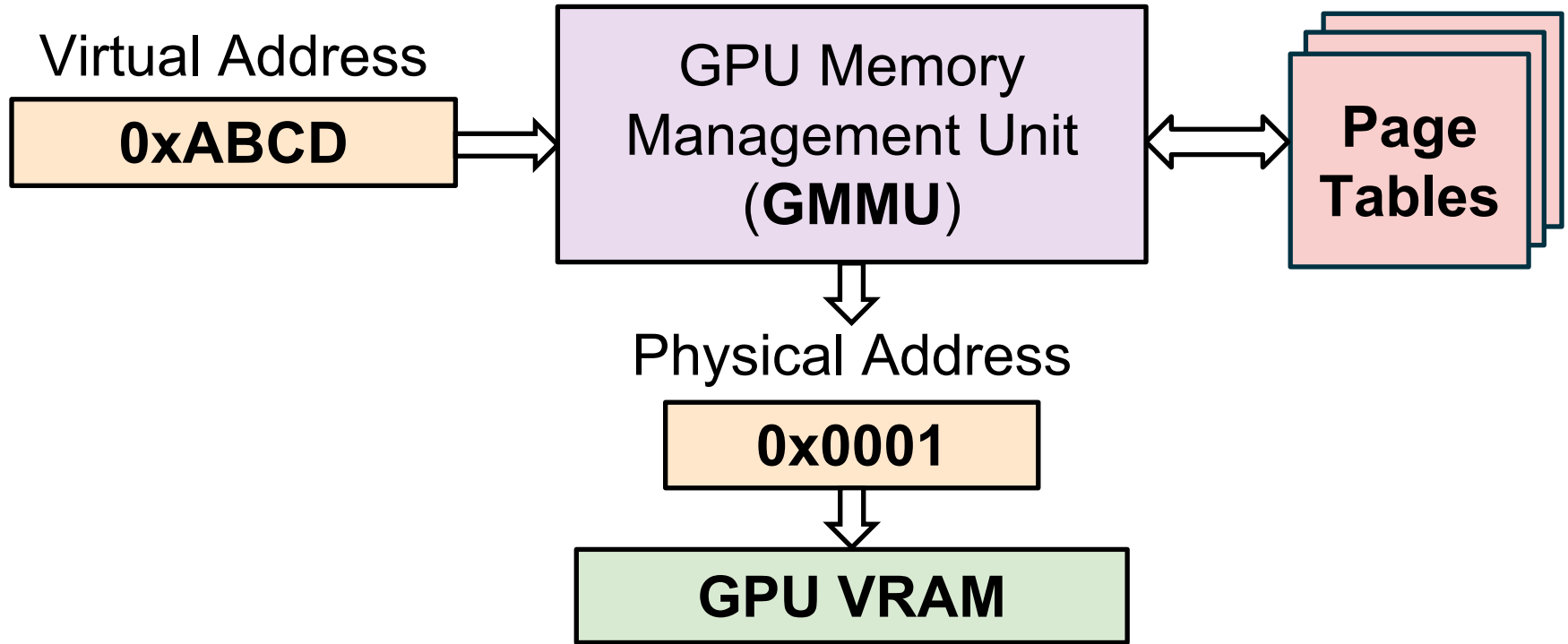


20+ Product Lines at Risk!

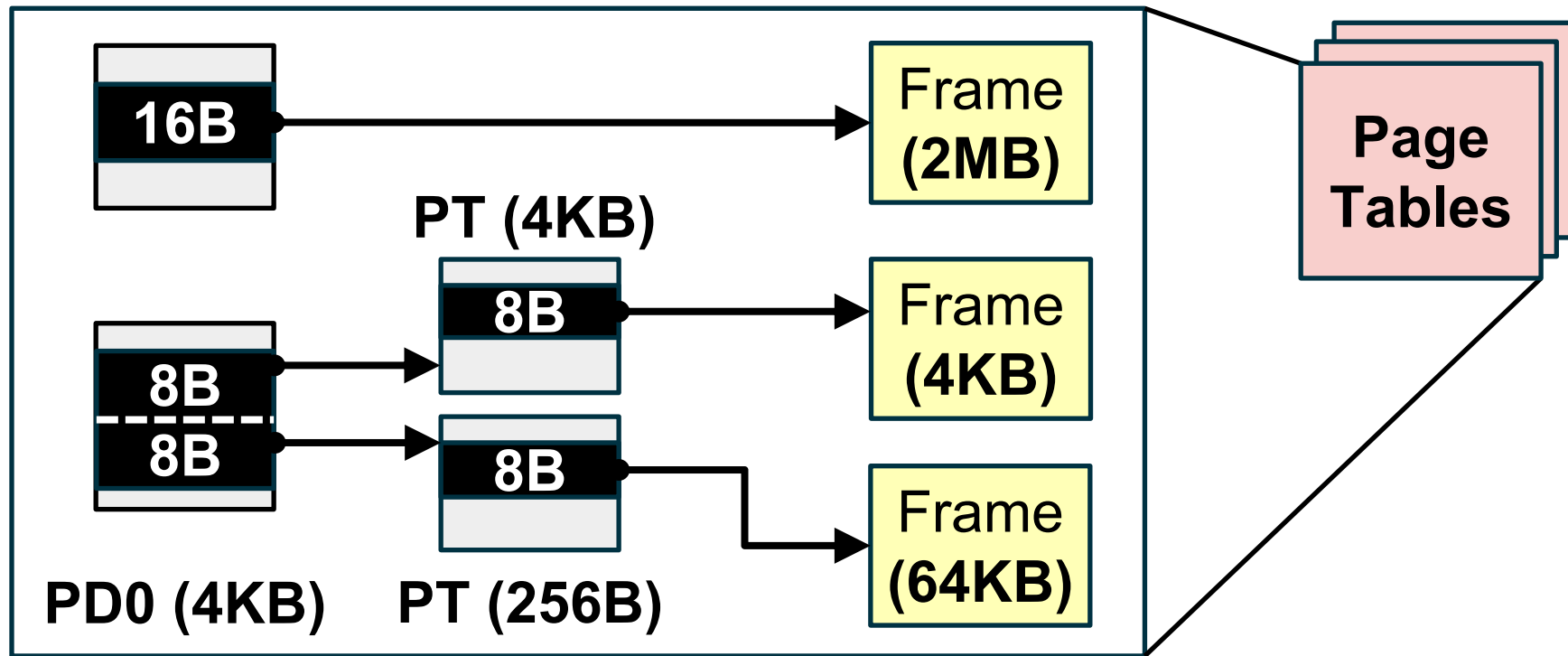


Background

Page Tables in GPU

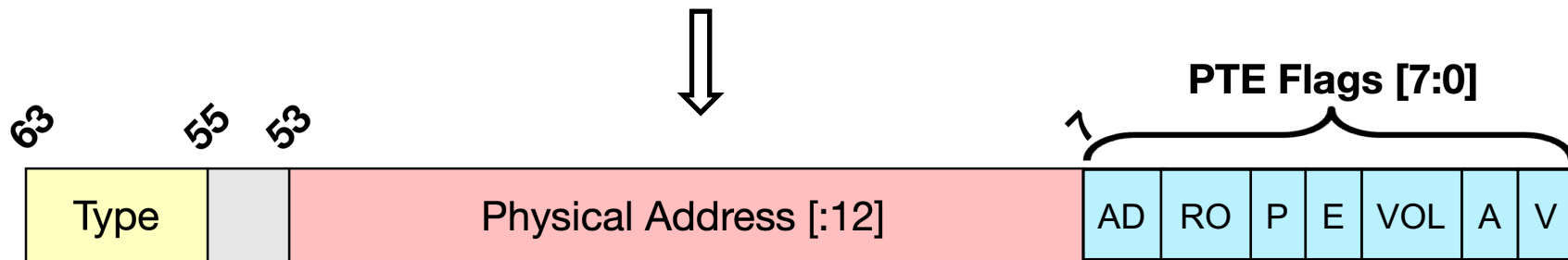


Page Tables in GPU



Page Tables in GPU: PTEs

We target the physical address bits.



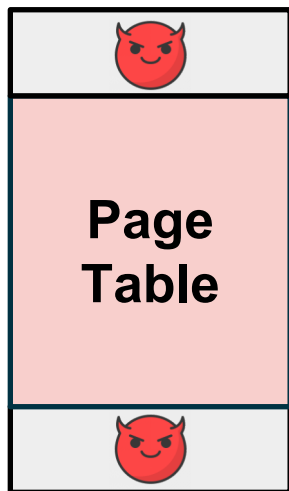
64-bit PTE Format

Page Table Tampering

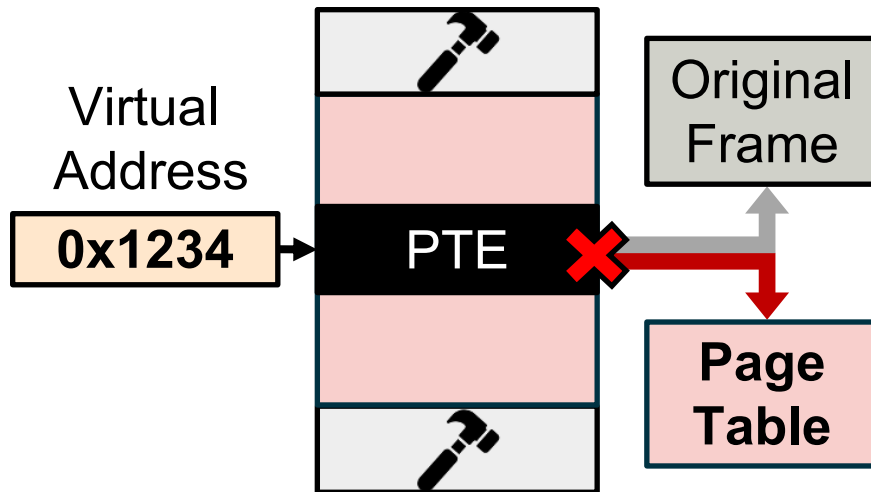
No need to thank me



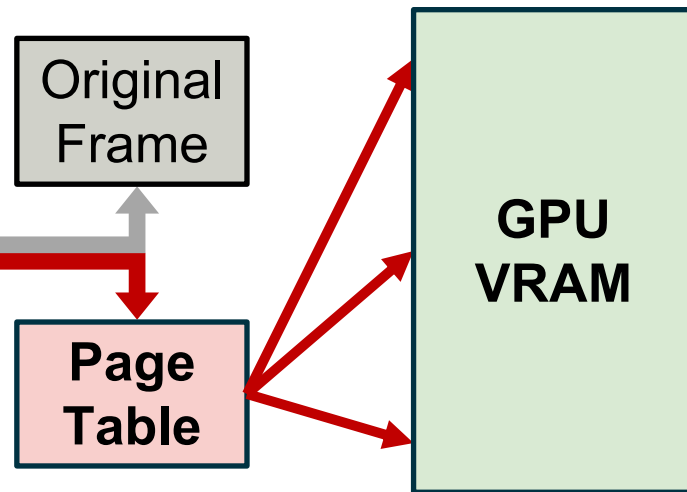
Step 1:
Neighbor PT & Aggressor



Step 2:
Corrupt PTE



Step 3:
Place PT in New Dest.



Page Table Tampering



From their own words (Blackhat '15):

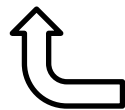
Mark was more clever: He simply put the system under memory pressure - when backed into a corner, the OS behaves nicely.

Page Table Tampering



From their own words (Blackhat '15):

Mark was more clever: He simply put the system under memory pressure - when backed into a corner, the OS behaves nicely.

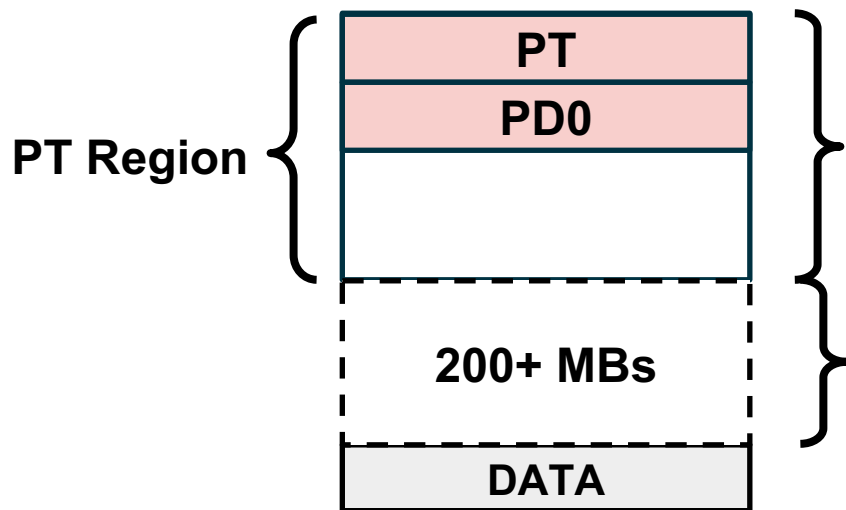


Can't Fill GPU VRAM with PTE like on Linux

Possible somehow. I spent a few afternoons fumbling around in the Linux physical page allocator. Not very fun code.

Where are Page Tables?

GPU Page Table Placements (2MB PT Region)



Challenge: Naively Filling PT Region
Requires Allocating **256 GB** of Memory

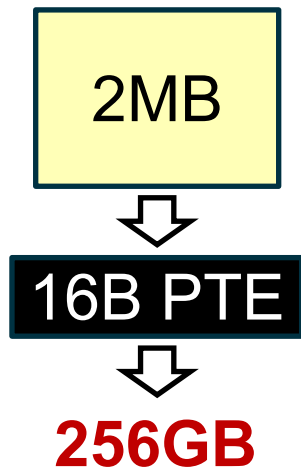
Initial PT Region is
Far Away (200+ MB) from User Data

Challenges For Page Table Massaging

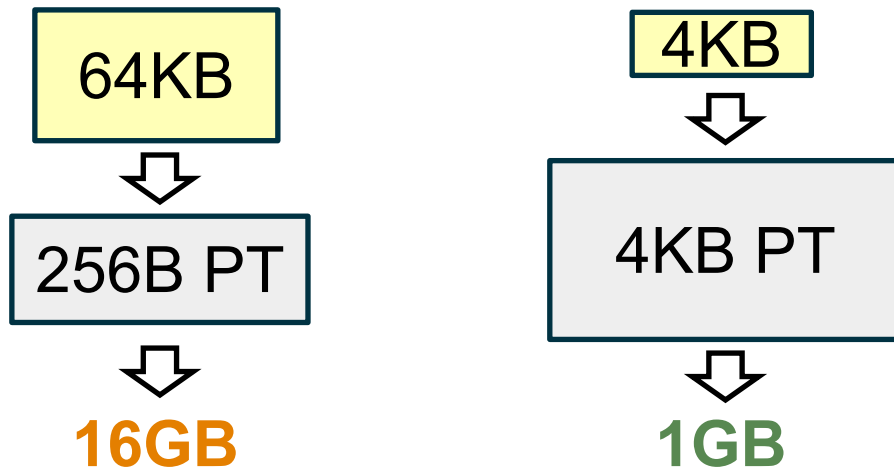
1 Memory Inefficiency of Page Table Allocations

Q1. How to Get More PT Regions?

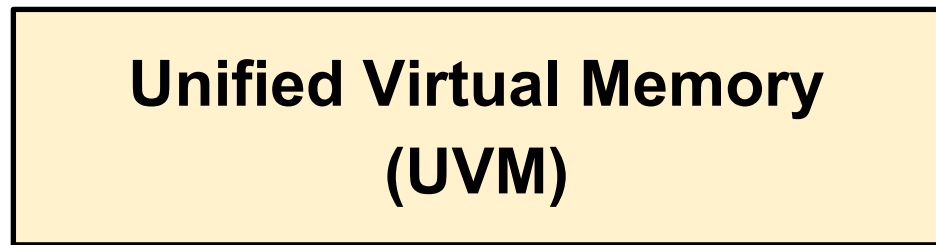
cudaMalloc()



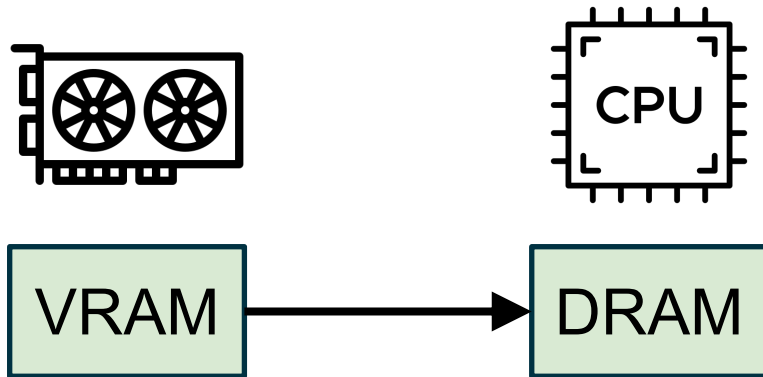
cudaMallocManaged()



Q1. How to Get More PT Regions?



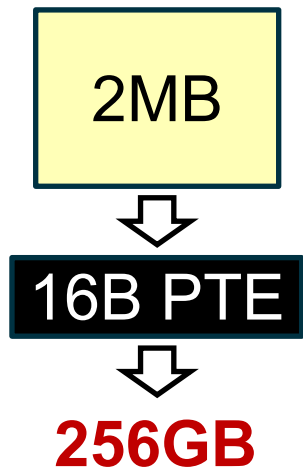
`cudaMallocManaged()`



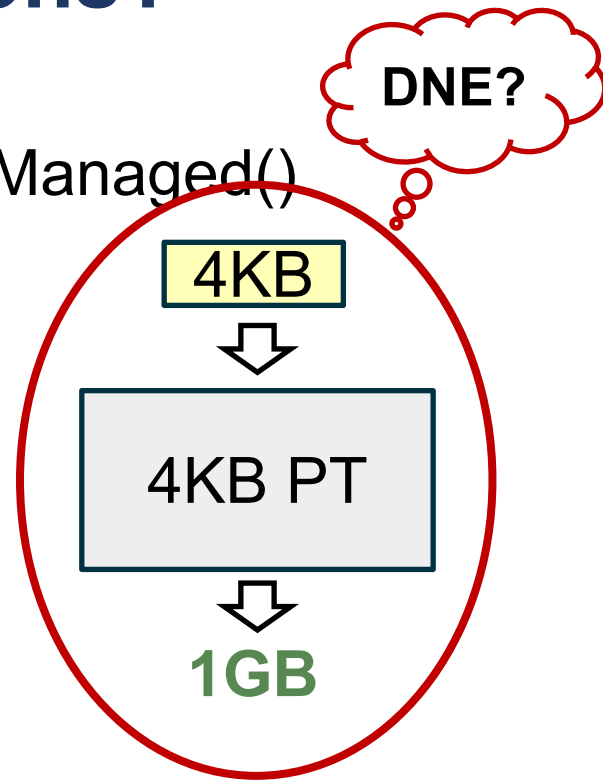
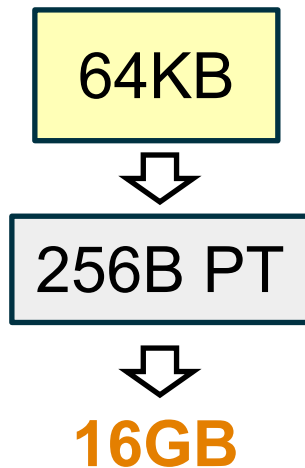
CPU as Swap

Q1. How to Get More PT Regions?

cudaMalloc()



cudaMallocManaged()

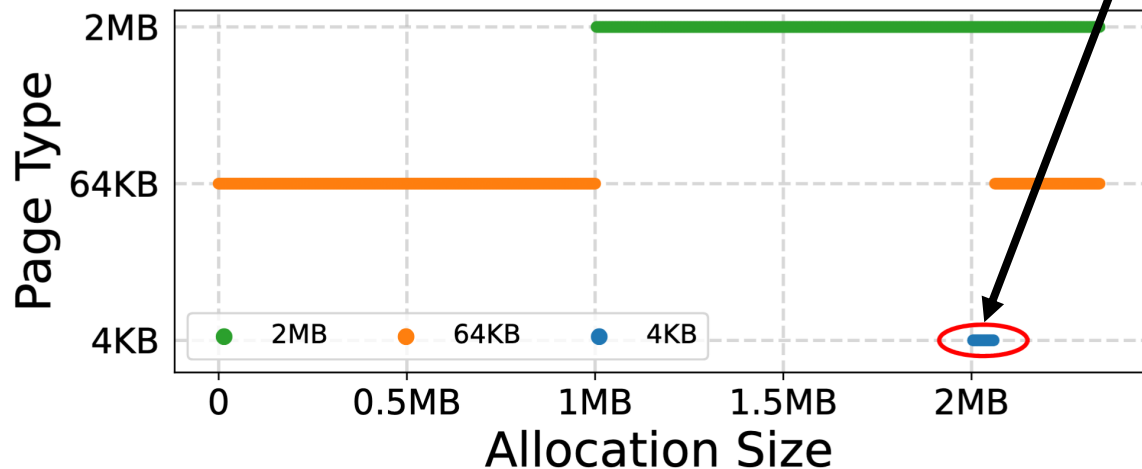


Getting 4KB Pages! (They thought it DNE...)

`cudaMallocManaged(&ptr, 2 MB + 4 KB)`

Getting 4KB Pages!

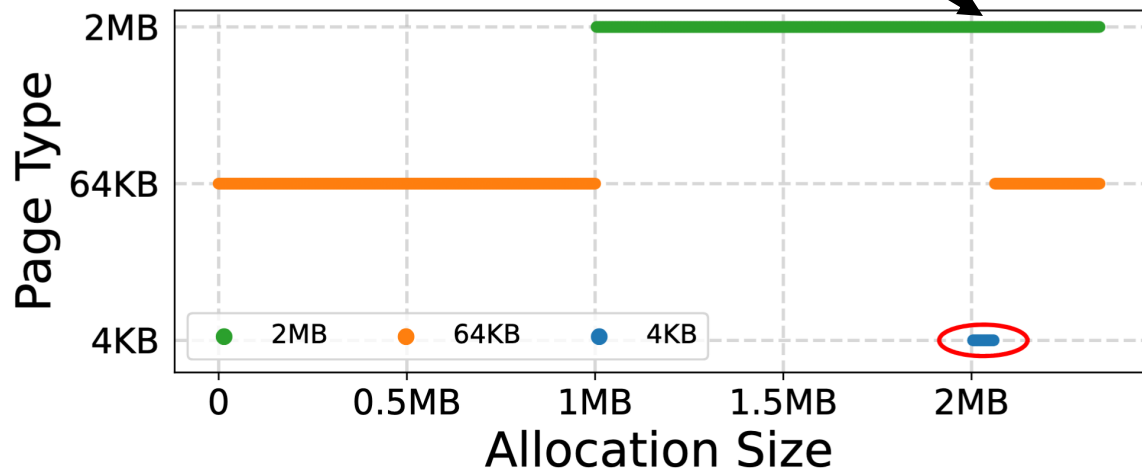
`cudaMallocManaged(&ptr, 2 MB + 4 KB)`



```
241-->PT@0x000c036000
      0-->4KB-Page@0x001cc00000
243-->PT@0x000c037000
      0-->4KB-Page@0x001cc01000
245-->PT@0x000c038000
      0-->4KB-Page@0x001cc02000
```

Getting 4KB Pages!

cudaMallocManaged(&ptr, 2 MB[?] + 4 KB)

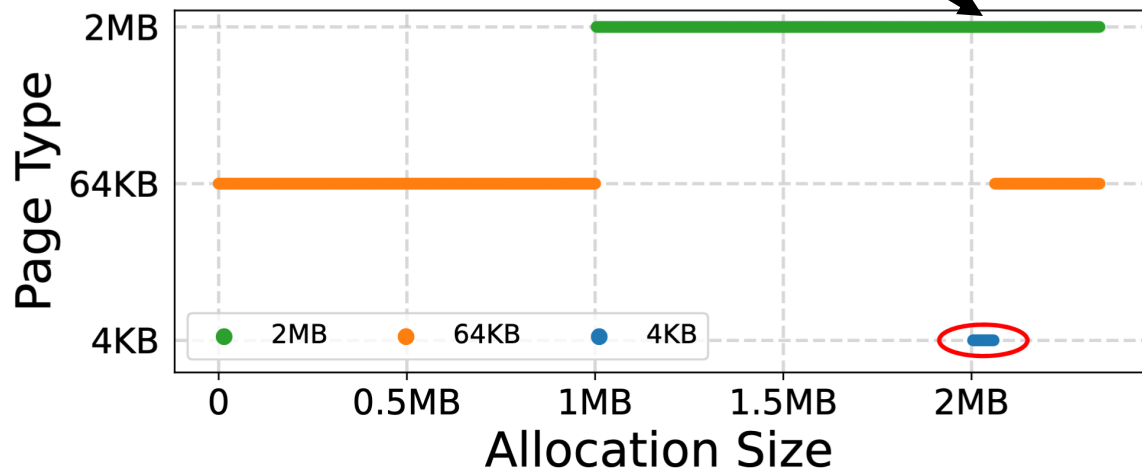


```
241-->PT@0x000c036000
      0-->4KB-Page@0x001cc00000
243-->PT@0x000c037000
      0-->4KB-Page@0x001cc01000
245-->PT@0x000c038000
      0-->4KB-Page@0x001cc02000
```

Getting 4KB Pages!

I am Lazy Loaded!

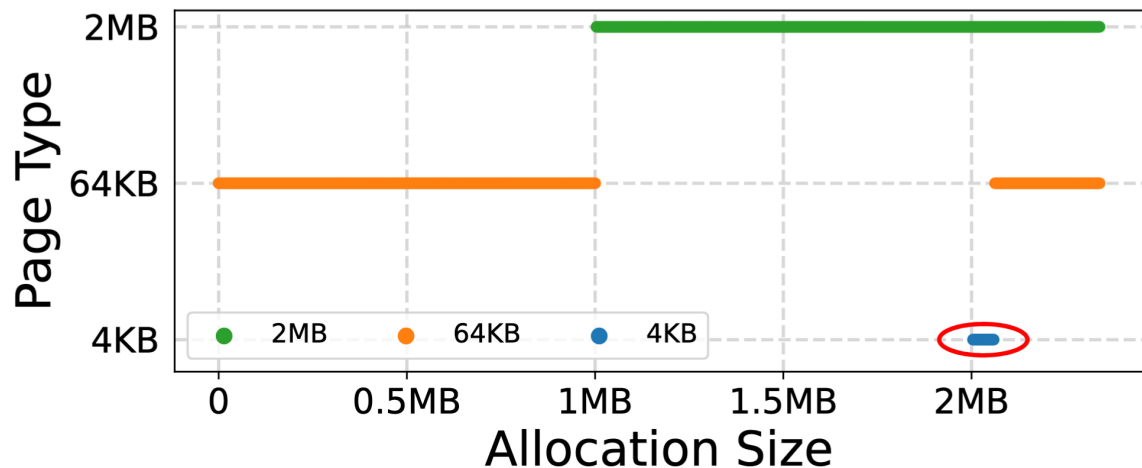
cudaMallocManaged(&ptr, 2 MB + 4 KB)



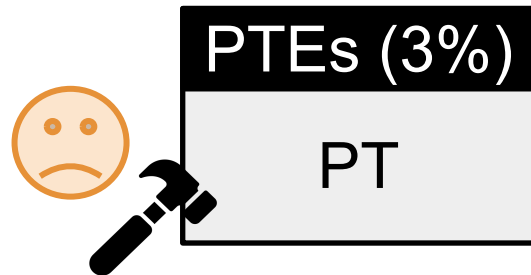
```
241-->PT@0x000c036000
      0-->4KB-Page@0x001cc00000
243-->PT@0x000c037000
      0-->4KB-Page@0x001cc01000
245-->PT@0x000c038000
      0-->4KB-Page@0x001cc02000
```

Q2. How to Get Dense PT Regions?

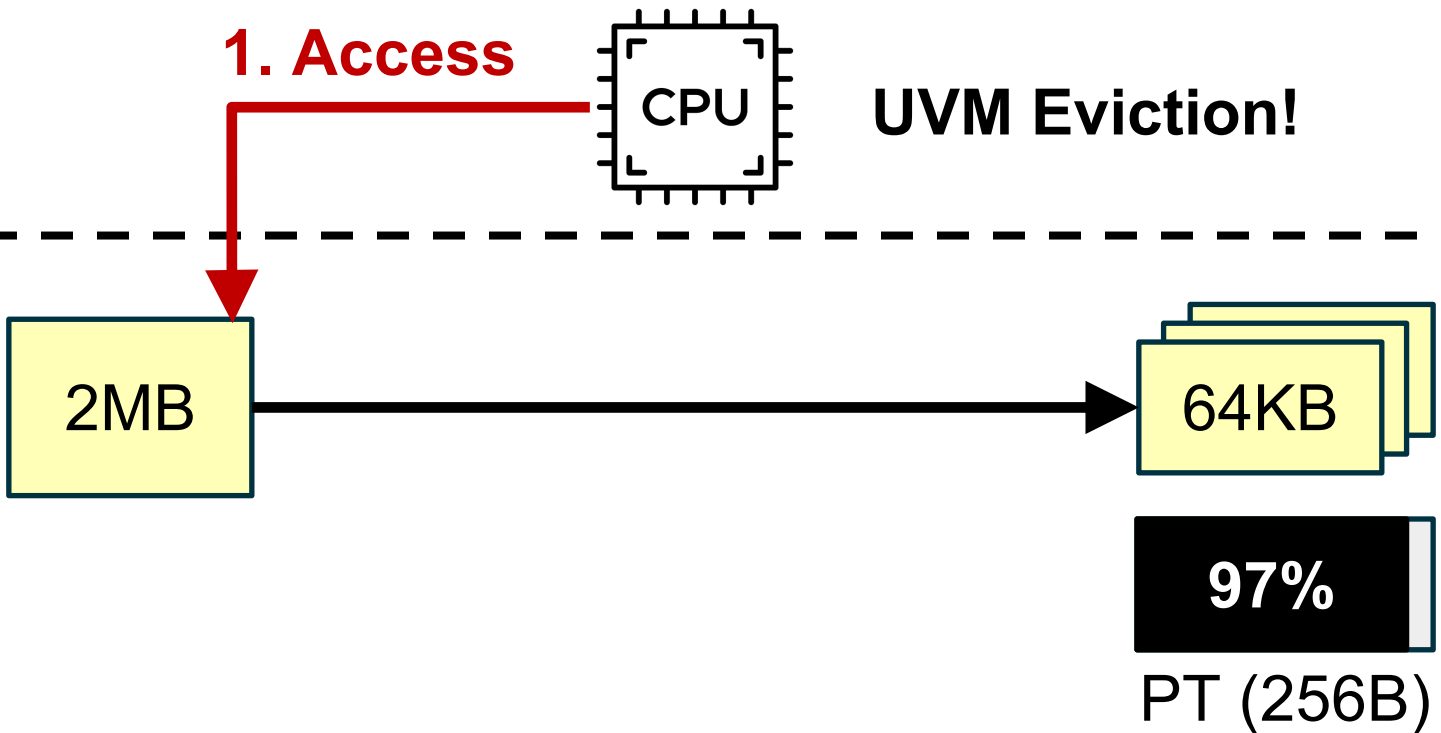
4KB PT can efficiently allocate PT Regions, **but...**



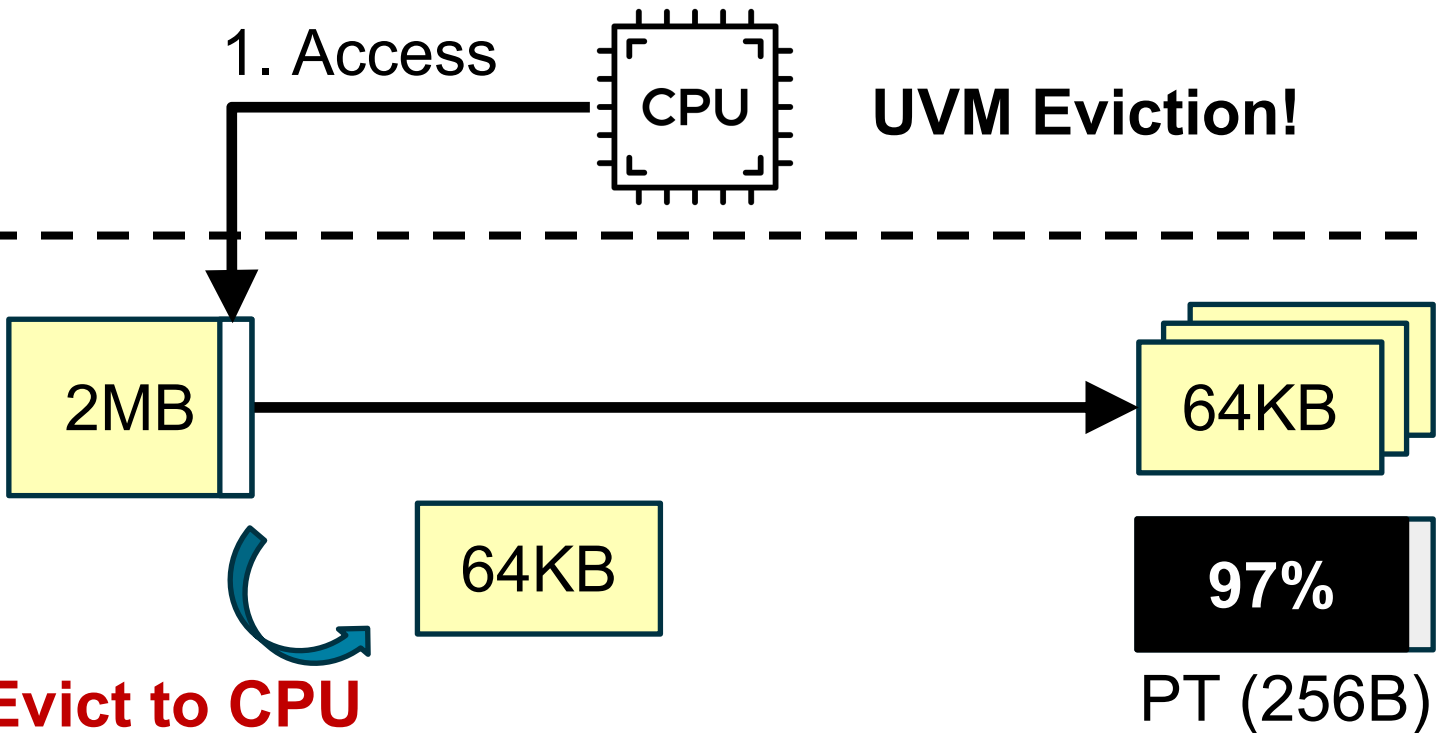
4KB PT is Sparse...
→ Hard to Hit PTEs



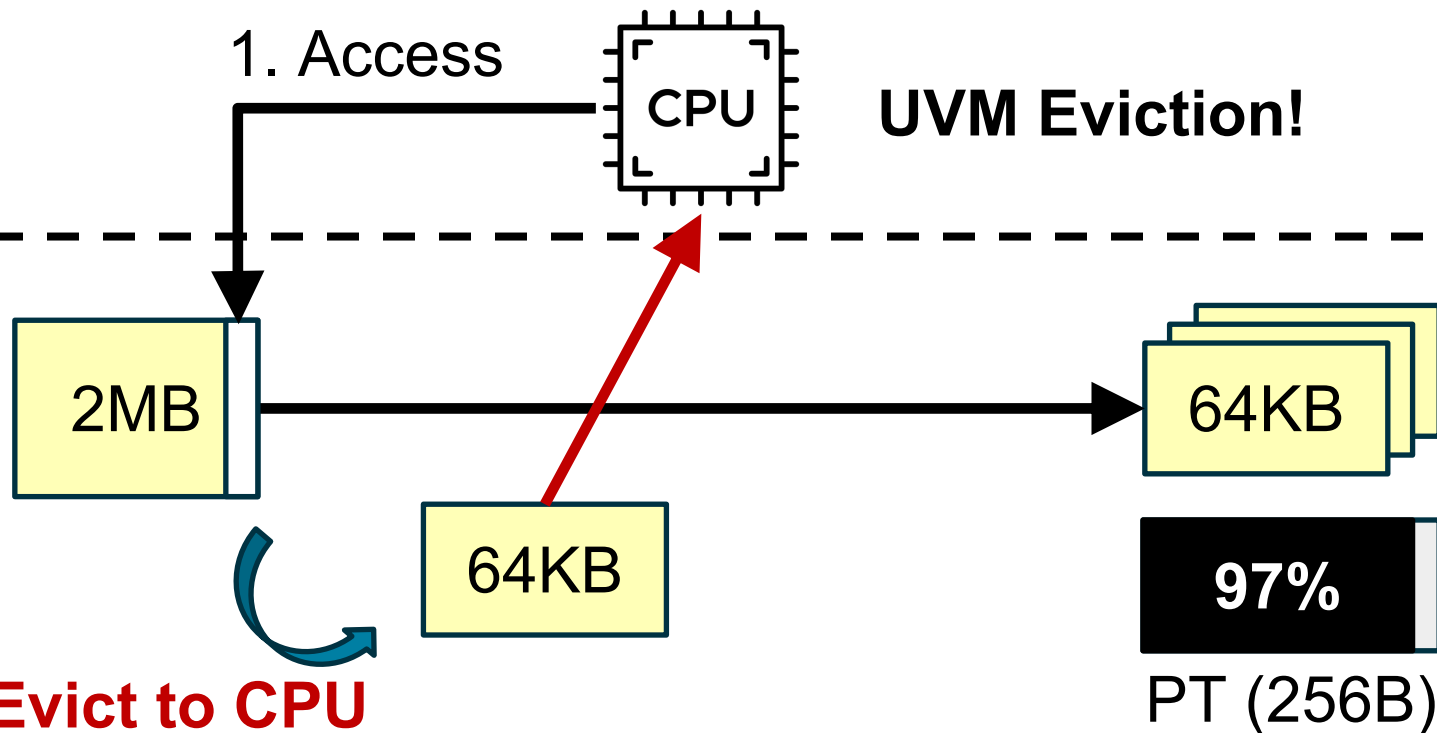
Q2. How to Get Dense PT Regions?



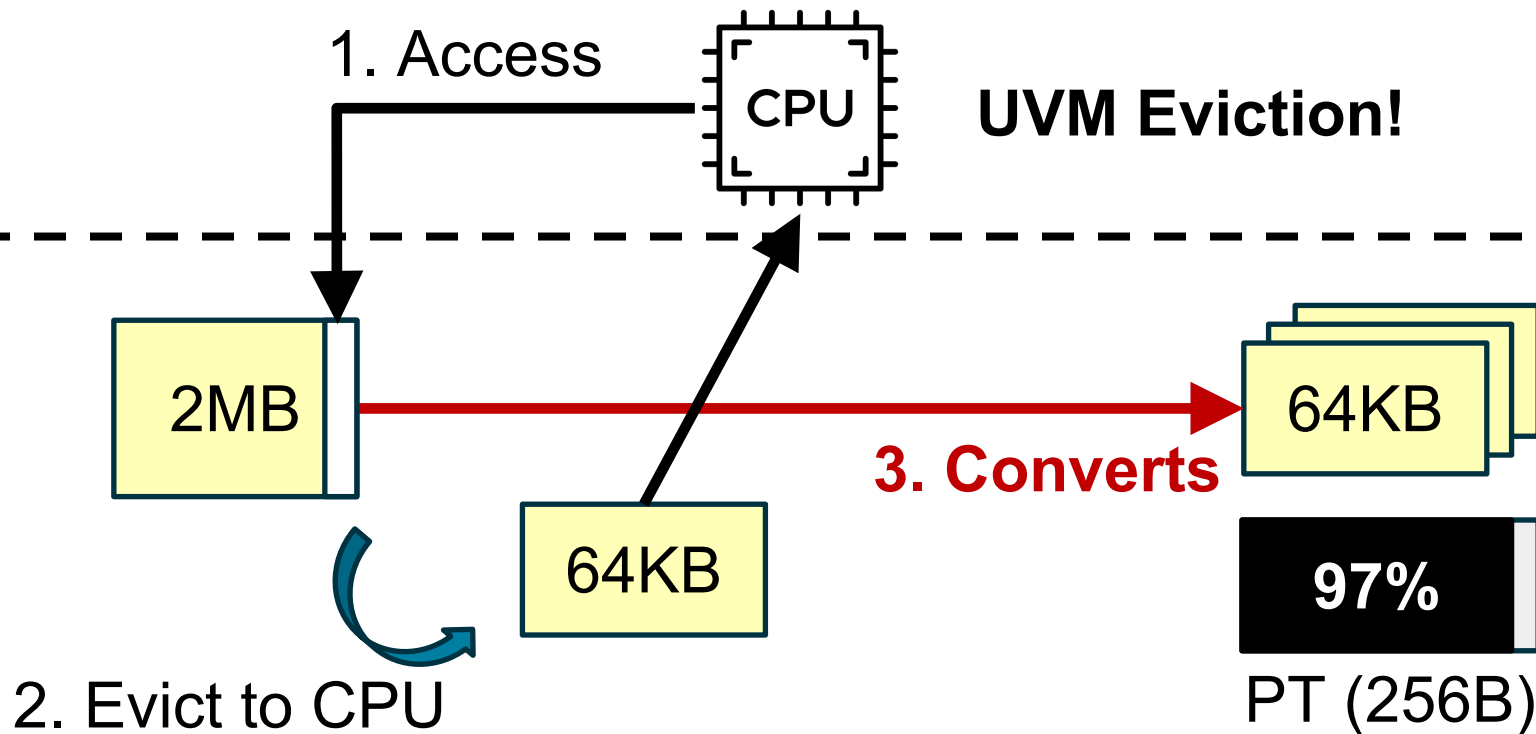
Q2. How to Get Dense PT Regions?



Q2. How to Get Dense PT Regions?



Q2. How to Get Dense PT Regions?





Before Eviction

243-->PD0@0x000c035000

48-->2MB-Page@0x001cc00000

After Eviction

243-->PD0@0x000c035000

48-->PT@0x000c022100

0-->64KB-Page@0x001cc00000

1-->64KB-Page@0x001cc10000

2-->64KB-Page@0x001cc20000

3-->64KB-Page@0x001cc30000



Before Eviction

243-->PD0@0x000c035000

48-->2MB-Page@0x001cc00000

After Eviction

243-->PD0@0x000c035000

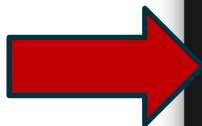
48-->PT@0x000c022100

0-->64KB-Page@0x001cc00000

1-->64KB-Page@0x001cc10000

2-->64KB-Page@0x001cc20000

3-->64KB-Page@0x001cc30000



Challenges For Page Table Massaging

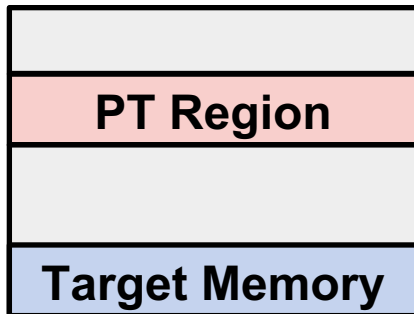
 1 Efficient & Dense PT Region Allocation Technique!

Challenges For Page Table Massaging

 **1 Efficient & Dense PT Region Allocation Technique!**

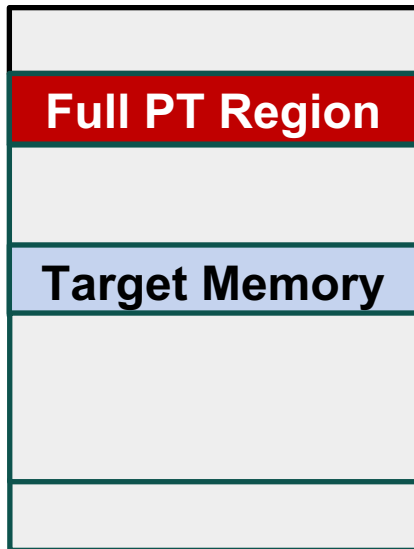
 **2 Lack of Control Over GPU PT Region Placement.**

How to Massage PTEs?



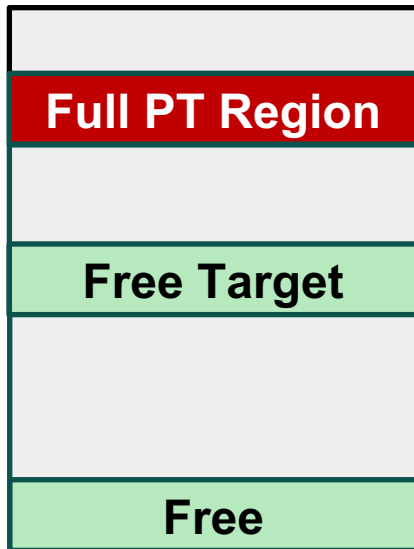
Observation:
PT Region and User Data
use the SAME memory pool

How to Massage PTEs?



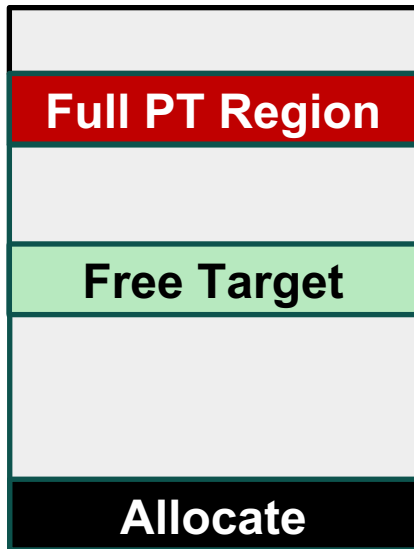
Observation:
PT Region and User Data
use the SAME memory pool

How to Massage PTEs?



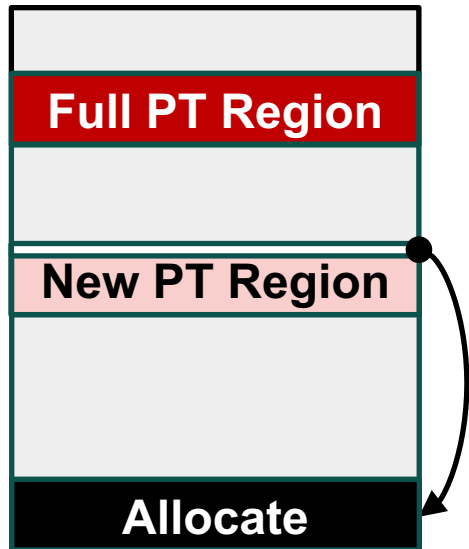
Observation:
PT Region and User Data
use the SAME memory pool

How to Massage PTEs?



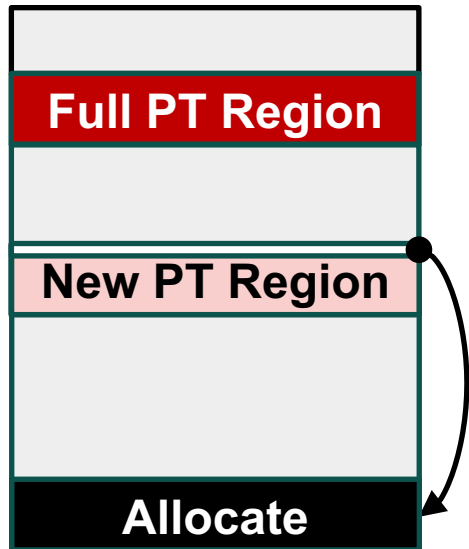
Observation:
PT Region and User Data
use the SAME memory pool

How to Massage PTEs?



Observation:
PT Region and User Data
use the SAME memory pool

How to Massage PTEs?



Observation:
PT Region and User Data
use the SAME memory pool

Question:
How do we know when PT Region is full?

Challenges For Page Table Massaging

 1 Efficient & Dense PT Region Allocation Technique!

 2 Exhaust VRAM to Force Placement!

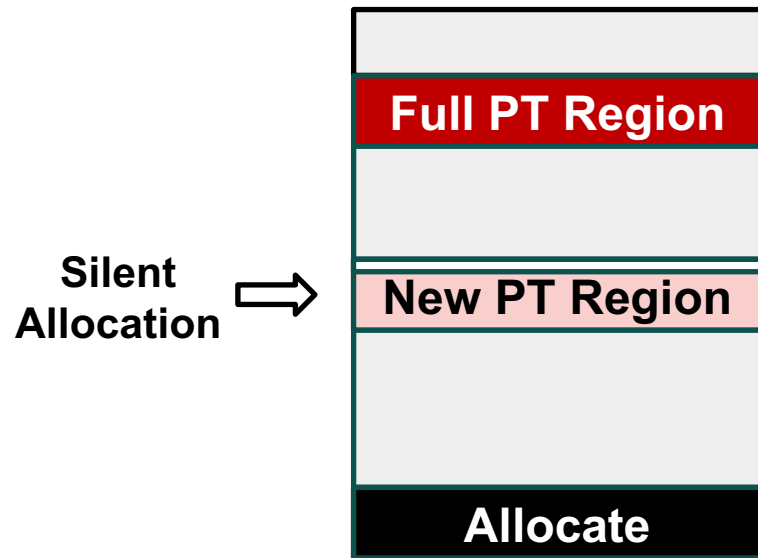
Challenges For Page Table Massaging

 1 Efficient & Dense PT Region Allocation Technique!

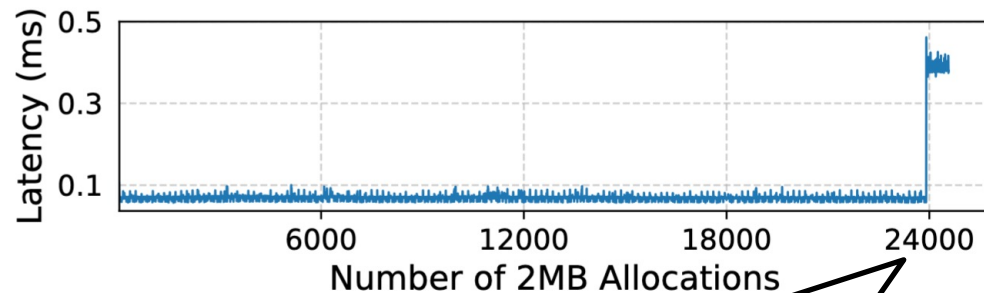
 2 Exhaust VRAM to Force Placement!

 3 PT Region Allocation is Invisible to Attacker.

Identify PT Region Allocations



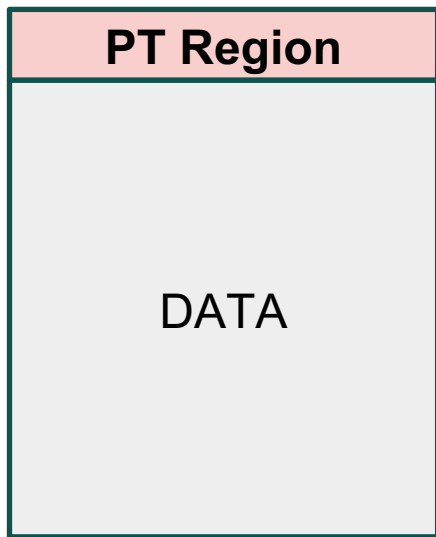
We leverage **timing side-channel** in **Unified Virtual Memory (UVM)**



Allocations Above Memory Limit
are **Evicted to CPU**

Eviction Timing Side-channel!

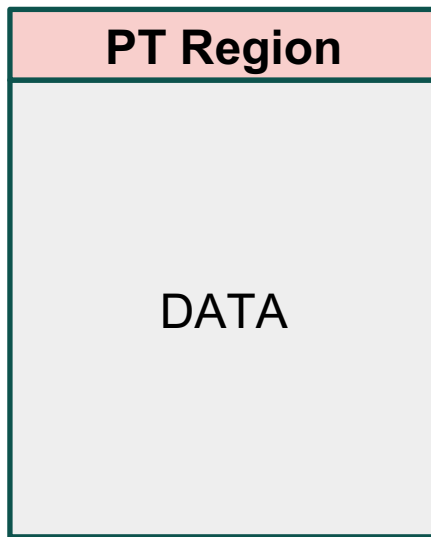
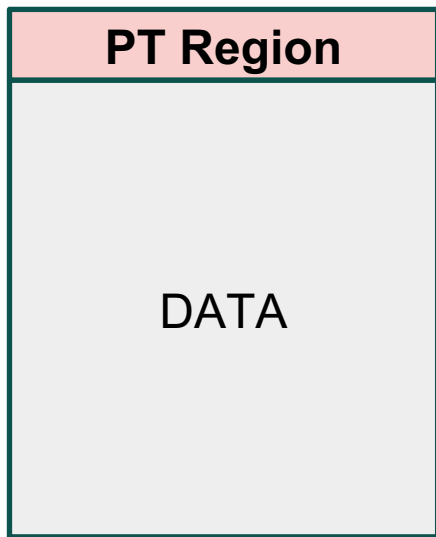
Fill GPU memory full



Eviction Timing Side-channel!

Fill GPU memory full

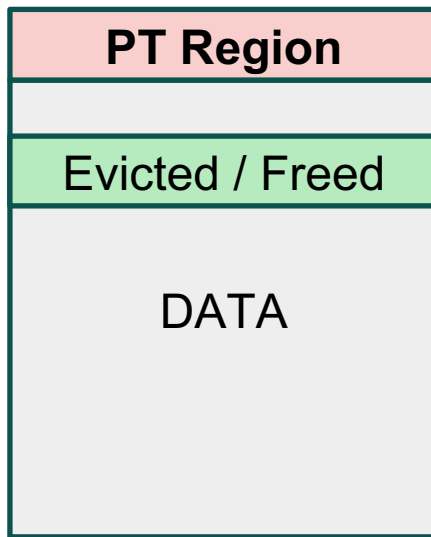
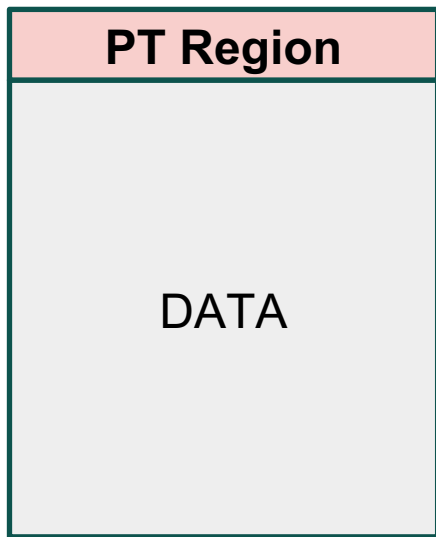
Repeat Evict and allocate
(Low Latency)



Eviction Timing Side-channel!

Fill GPU memory full

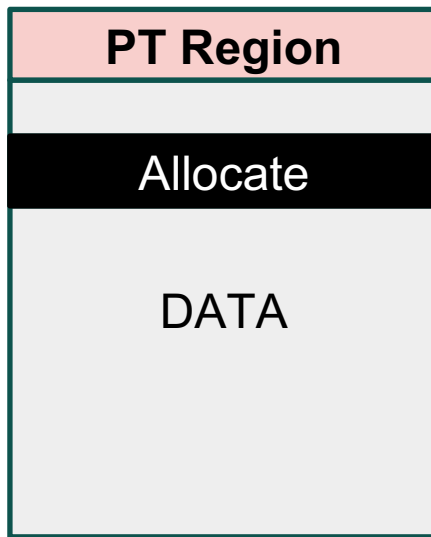
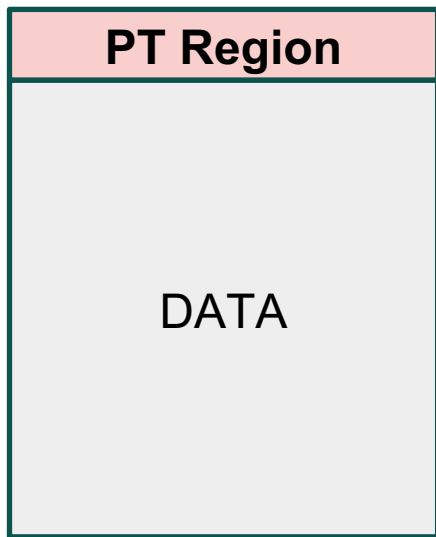
Repeat Evict and allocate
(Low Latency)



Eviction Timing Side-channel!

Fill GPU memory full

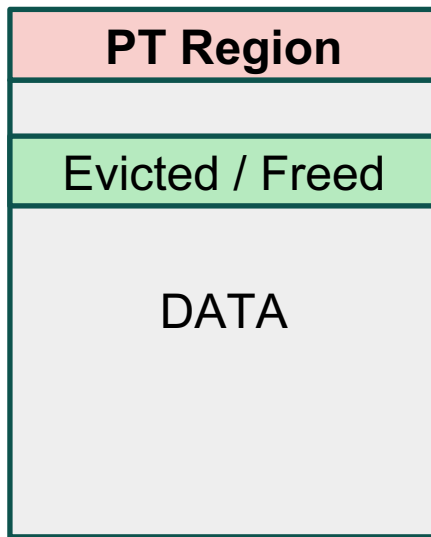
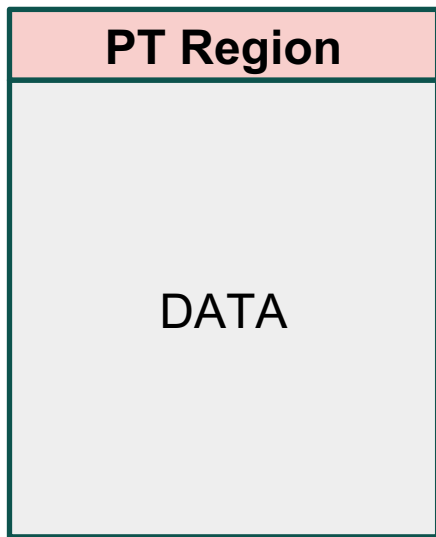
Repeat Evict and allocate
(Low Latency)



Eviction Timing Side-channel!

Fill GPU memory full

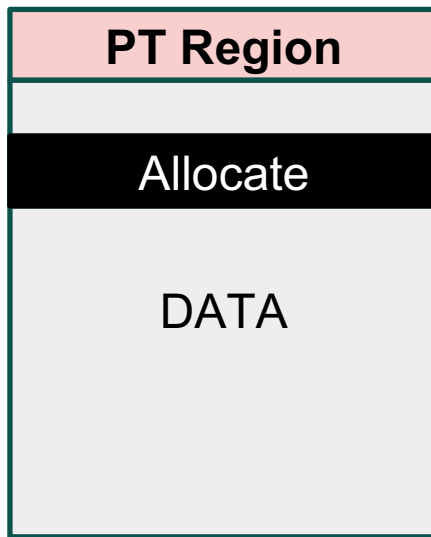
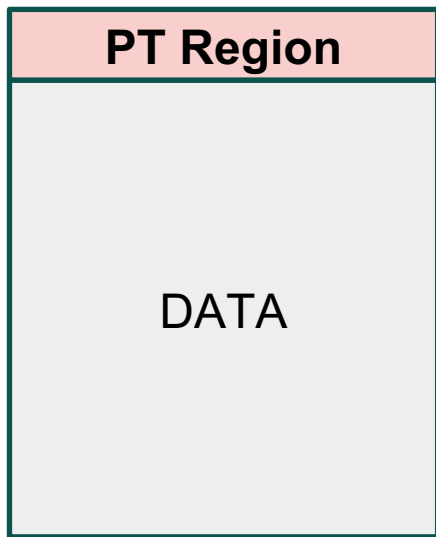
Repeat Evict and allocate
(Low Latency)



Eviction Timing Side-channel!

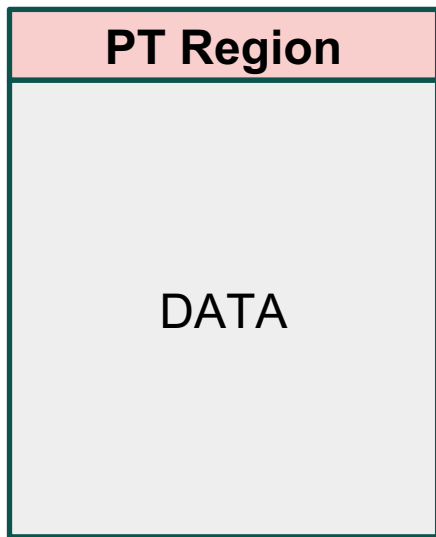
Fill GPU memory full

Repeat Evict and allocate
(Low Latency)

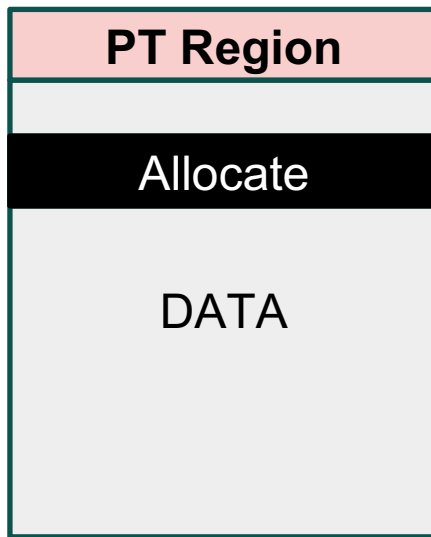


Eviction Timing Side-channel!

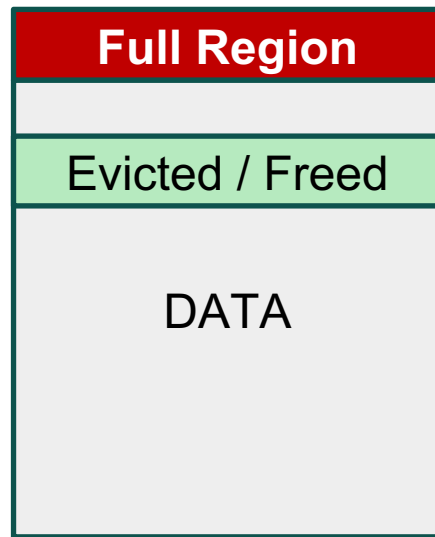
Fill GPU memory full



Repeat Evict and allocate
(Low Latency)

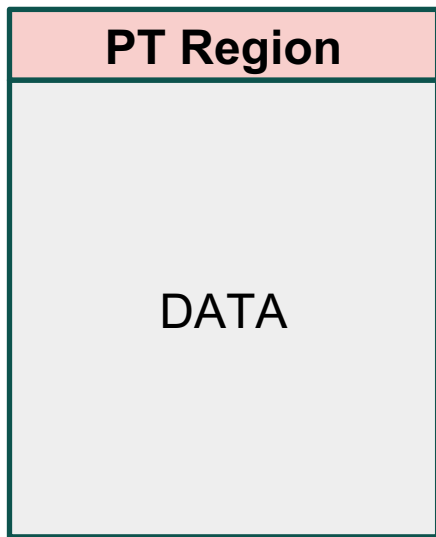


New PT region needed
(High Latency!)

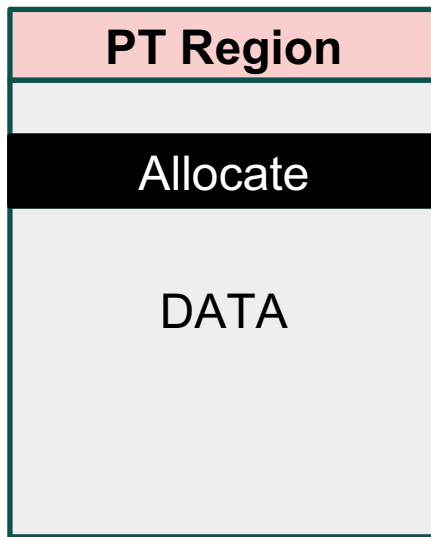


Eviction Timing Side-channel!

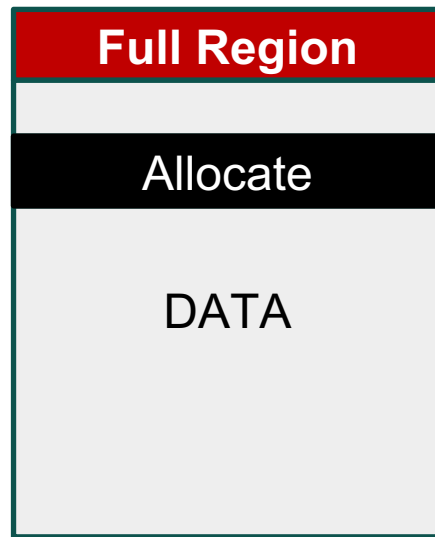
Fill GPU memory full



Repeat Evict and allocate
(Low Latency)

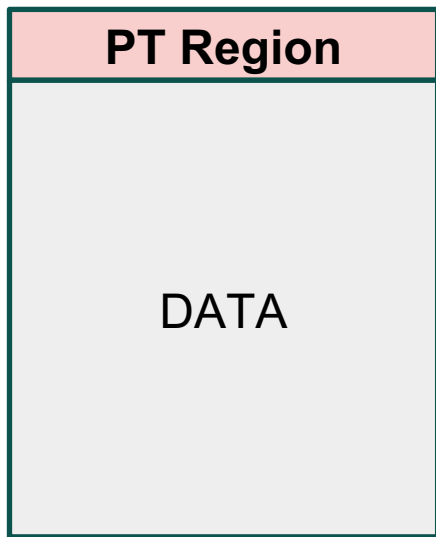


New PT region needed
(High Latency!)

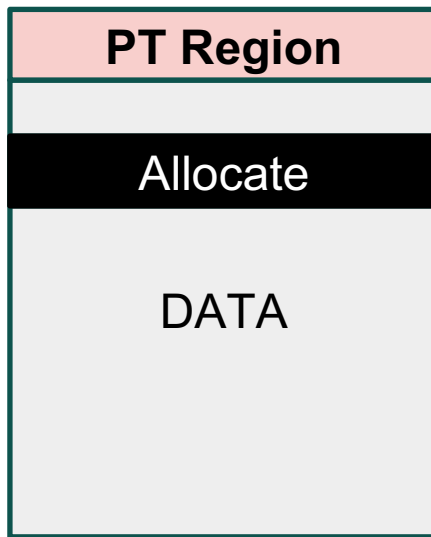


Eviction Timing Side-channel!

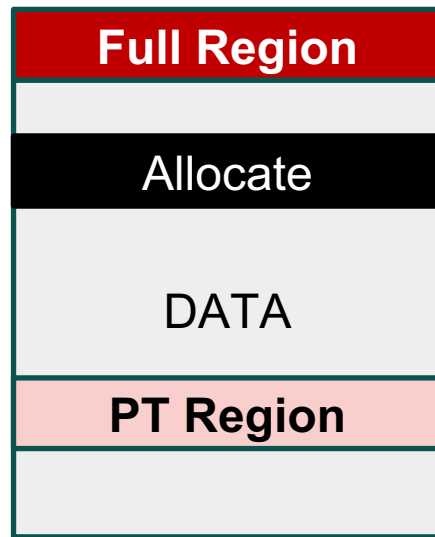
Fill GPU memory full



Repeat Evict and allocate
(Low Latency)

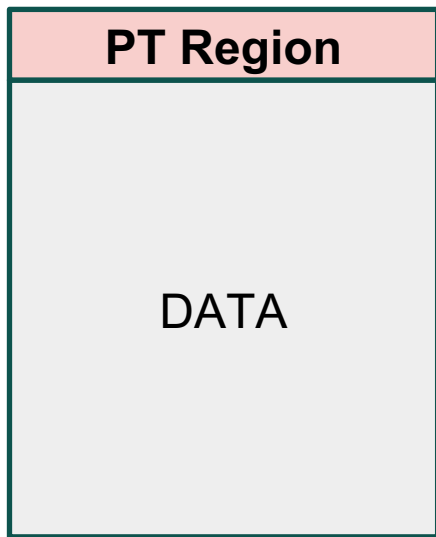


New PT region needed
(High Latency!)

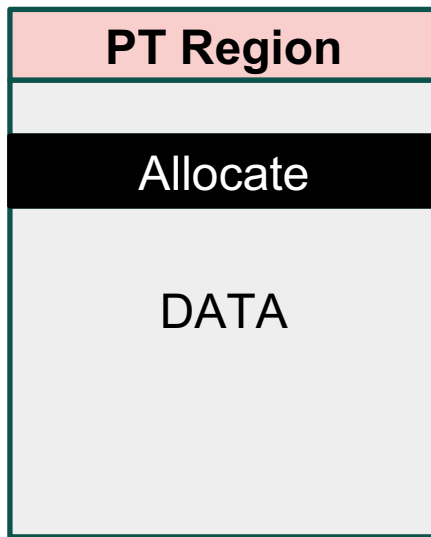


Eviction Timing Side-channel!

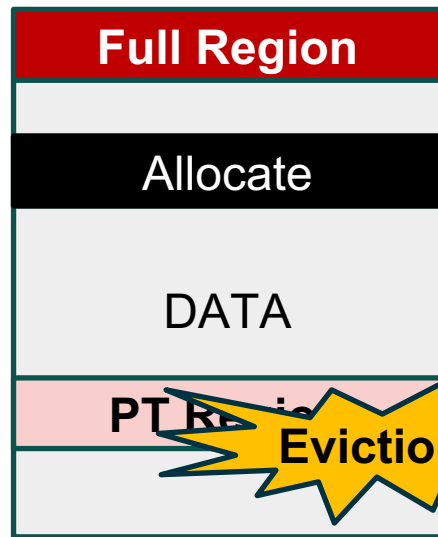
Fill GPU memory full



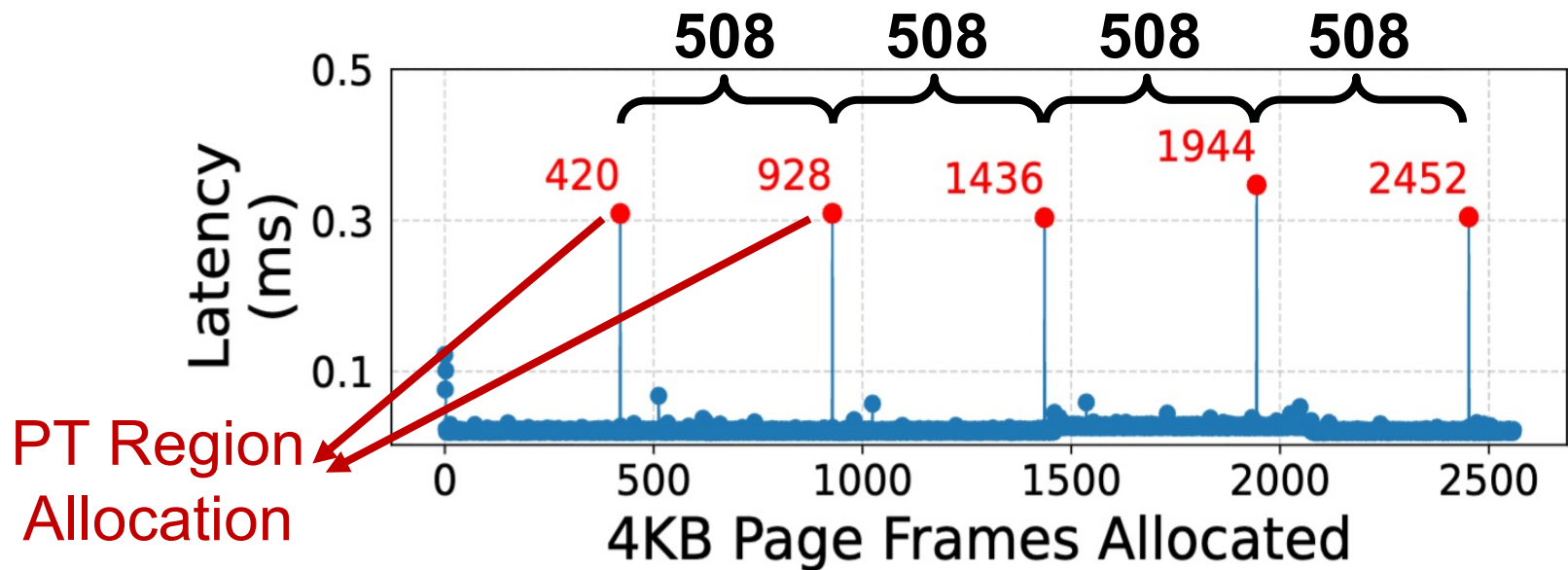
Repeat Evict and allocate
(Low Latency)



New PT region needed
(High Latency!)



Eviction Timing Side-channel!



We can **deterministically** predict PT Region Allocations.

Challenges For Page Table Massaging

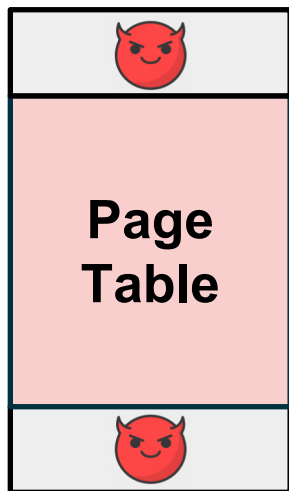
 1 Efficient & Dense PT Region Allocation Technique!

 2 Exhaust VRAM to Force Placement!

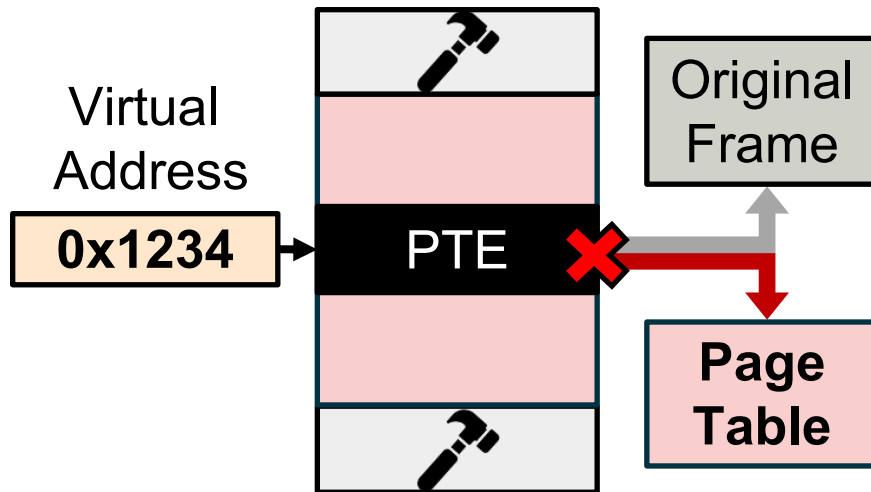
 3 Timing Side-channel to Detect Allocation!

Page Table Tampering

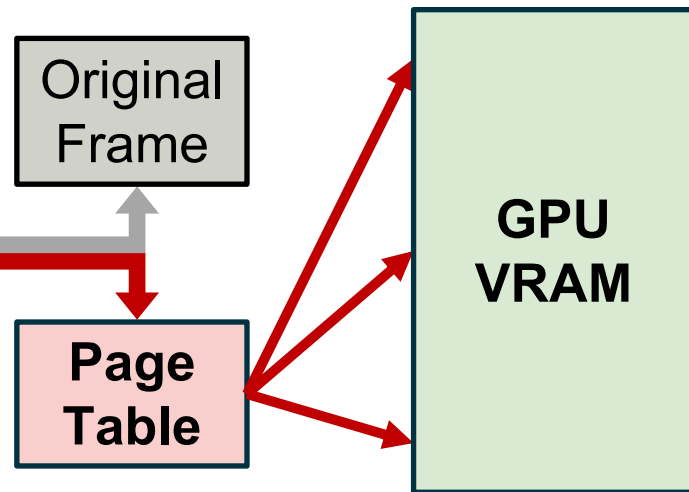
Step 1:
Neighbor PT & Aggressor



Step 2:
Corrupt PTE



Step 3:
Place PT in New Dest.



GPUBreach can **Practically & Reliably Tamper GPU Page Tables** and enable downstream privilege escalation.



NVIDIA

Works on open-source driver versions from 2023-26.
Tested on RTX A6000 GPU.



Exploits!

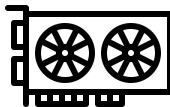
Exploit 1: Arbitrary Read on GPU

Arbitrary Read:
What can we steal?

Exploit 1: Arbitrary Read on GPU

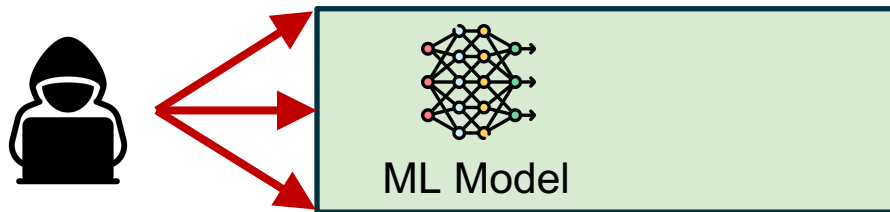


Attacker



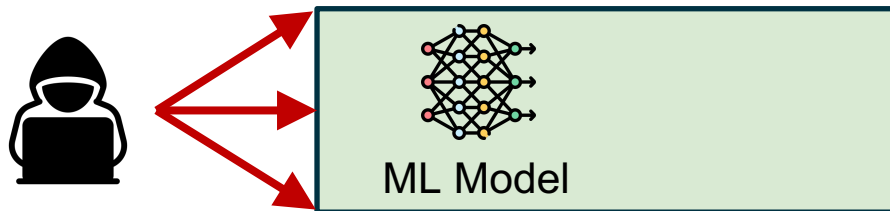
Arbitrary Read:
What can we steal?

Exploit 1: Arbitrary Read on GPU



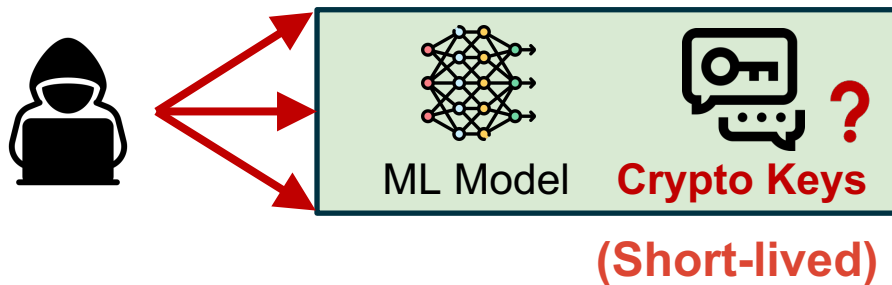
	Variant
Llama2 (7B)	Meta* Nous Meta, Chat
Llama3 (8B)	Meta* Nous Meta, Instruct
Mistral (7B)	v1.0* Instruct v1.0 OpenHermes 2.5
Gemma (7B)	Google* Google, Instruct

Exploit 1: Arbitrary Read on GPU

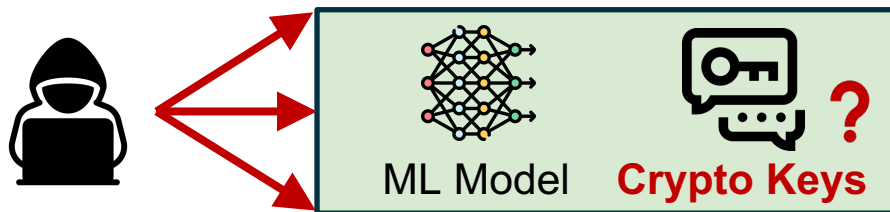


	Variant	Llama2	Llama3	Mistral	Gemma
Llama2 (7B)	Meta*	0.99	-0.20	-0.05	0.29
	Nous	0.99	-0.20	-0.05	0.29
	Meta, Chat	0.99	-0.20	-0.05	0.29
Llama3 (8B)	Meta*	0.06	1.00	0.07	-0.02
	Nous	0.06	1.00	0.07	-0.02
	Meta, Instruct	0.06	1.00	0.07	-0.02
Mistral (7B)	v1.0*	0.94	-0.18	1.00	0.23
	Instruct v1.0	0.94	-0.18	1.00	0.25
	OpenHermes 2.5	0.94	-0.18	1.00	0.23
Gemma (7B)	Google*	-0.04	-0.20	-0.08	1.00
	Google, Instruct	-0.03	-0.16	-0.04	0.97

Exploit 1: Arbitrary Read on GPU



Exploit 1: Arbitrary Read on GPU

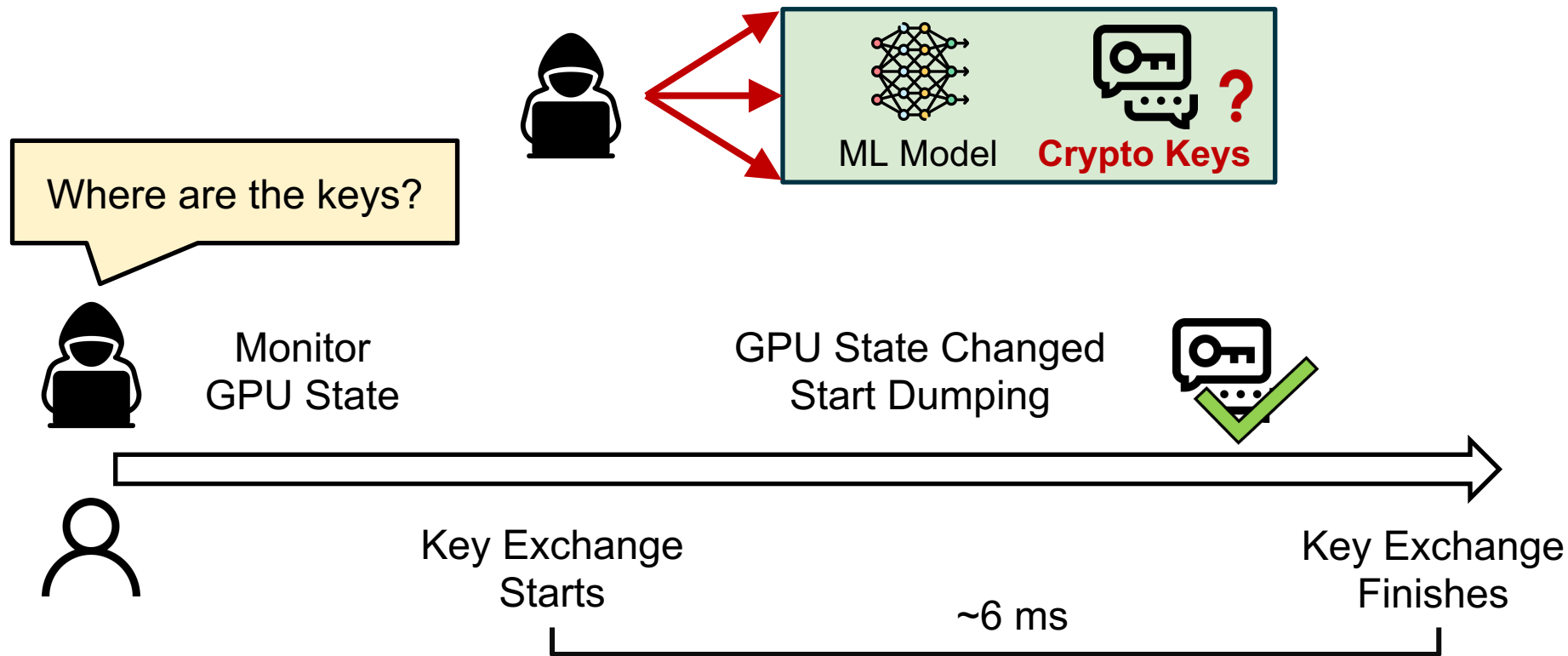


NVIDIA cuPQC

<https://developer.nvidia.com/cupqc>

NVIDIA cuPQC is an SDK of GPU-optimized cryptographic math libraries for building both classical and next-generation high-performance cryptographic applications.

Exploit 1: Arbitrary Read on GPU



Exploit 1: Arbitrary Read on GPU

Approach 1: Shared Library

open-quantum-
safe/**liboqs**



Attacker Key



0x1234

Victim Key



0x1234



Physical Location Equivalent

Approach 2: Victim Page Profiling

Obs. Memories are Zeroed on Free

0xFFFFFFFF

0xFFFFFFFF

0xFFFFFFFF

0xFFFFFFFF

Before

Exploit 1: Arbitrary Read on GPU

Approach 1: Shared Library

open-quantum-
safe/liboqs



Attacker Key



0x1234

Victim Key



0x1234



Physical Location Equivalent

Approach 2: Victim Page Profiling

Obs. Memories are Zeroed on Free

0xFFFFFFFF
0xFFFFFFFF
0xFFFFFFFF
0xFFFFFFFF

Before

0xFFFFFFFF
0x0
0xFFFFFFFF
0x0

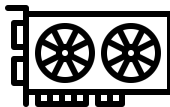
After



Exploit 1: Arbitrary Read on GPU



Attacker

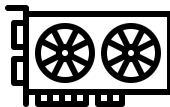


Arbitrary Read:
What can we steal?

Exploit 1: Arbitrary Read on GPU



Attacker

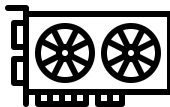


Leak **ML model weights** and **secret keys** from GPU memory

Exploit 2: Arbitrary Write on GPU



Attacker

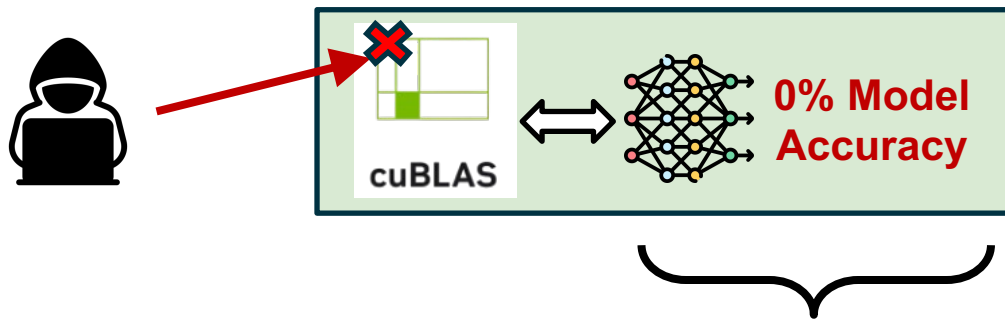


Victim

Leak **ML model weights** and **secret keys** from GPU memory

Arbitrary Write:
What can we tamper?

Exploit 2: Arbitrary Write on GPU



**Universal
Degradation**

But everything seems fine...

Exploit 2: Arbitrary Write on GPU

Offline Phase: Filter Critical Instructions

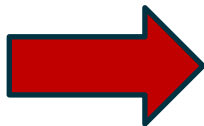
```
/* 0x00000110000000947, 0x003fde0003800000 */  
@P0 BRA 0x1d0 ;  
/* 0x00000000000007947, 0x003fde0003800000 */  
BRA 0xd0 ;  
...  
/* 0x00000000000007918, 0x000fc00000000000 */  
NOP;
```



Exploit 2: Arbitrary Write on GPU

Offline Phase: Filter Critical Instructions

Identify Kernels

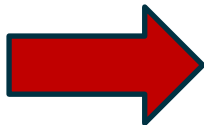


```
/* 0x000000110000000947, 0x003fde0003800000 */  
@P0 BRA 0x1d0 ;  
/* 0x000000000000007947, 0x003fde0003800000 */  
BRA 0xd0 ;  
...  
/* 0x000000000000007918, 0x000fc00000000000 */  
NOP;
```

Exploit 2: Arbitrary Write on GPU

Offline Phase: Filter Critical Instructions

Branch in SASS



```
/* 0x00000110000000947, 0x003fde0003800000 */
@P0 BRA 0x1d0 ;
/* 0x000000000000007947, 0x003fde0003800000 */
BRA 0xd0 ;
...
/* 0x000000000000007918, 0x000fc00000000000 */
NOP;
```

Exploit 2: Arbitrary Write on GPU

Offline Phase: Filter Critical Instructions

Goal: Find Branch that

- (1) Degrade models universally
- (2) ~0% Runtime Impact

```
/* 0x0000011000000947, 0x003fde0003800000 */
@P0 BRA 0x1d0 ;
/* 0x00000000000007947, 0x003fde0003800000 */
BRA 0xd0 ;
...
/* 0x00000000000007918, 0x000fc00000000000 */
NOP;
```

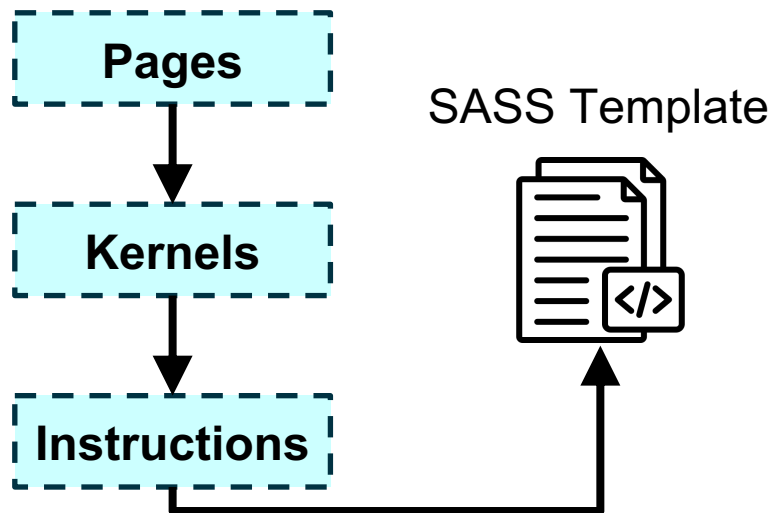
Exploit 2: Arbitrary Write on GPU

Offline Phase: Filter Critical Instructions

- Goal:** Find Branch that
- (1) Degrade models universally
 - (2) ~0% Runtime Impact

```
/* 0x00000011000000947, 0x003fde0003800000 */
@P0 BRA 0x1d0 ;
/* 0x00000000000007947, 0x003fde0003800000 */
BRA 0xd0 ;
...
/* 0x00000000000007918, 0x000fc00000000000 */
NOP;
```

Filter by this criteria at different granularity



Exploit 2: Arbitrary Write on GPU

Offline Phase:

Filter Critical Instructions

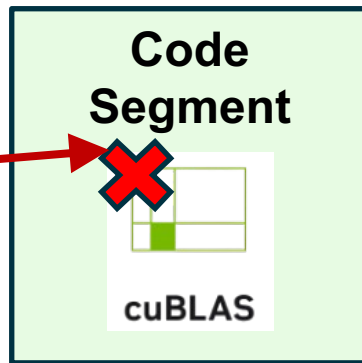
SASS Template



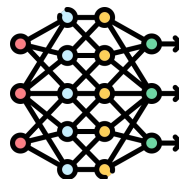
Online Phase:

Apply Template to Victim Code

PyTorch



0% Model Accuracy



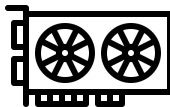
No

- (1) CPU-side Change
- (2) Runtime Change

Exploit 2: Arbitrary Write on GPU



Attacker



Victim

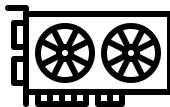
Leak **ML model weights** and **secret keys** from GPU memory

Arbitrary Write:
What can we tamper?

Exploit 2: Arbitrary Write on GPU



Attacker



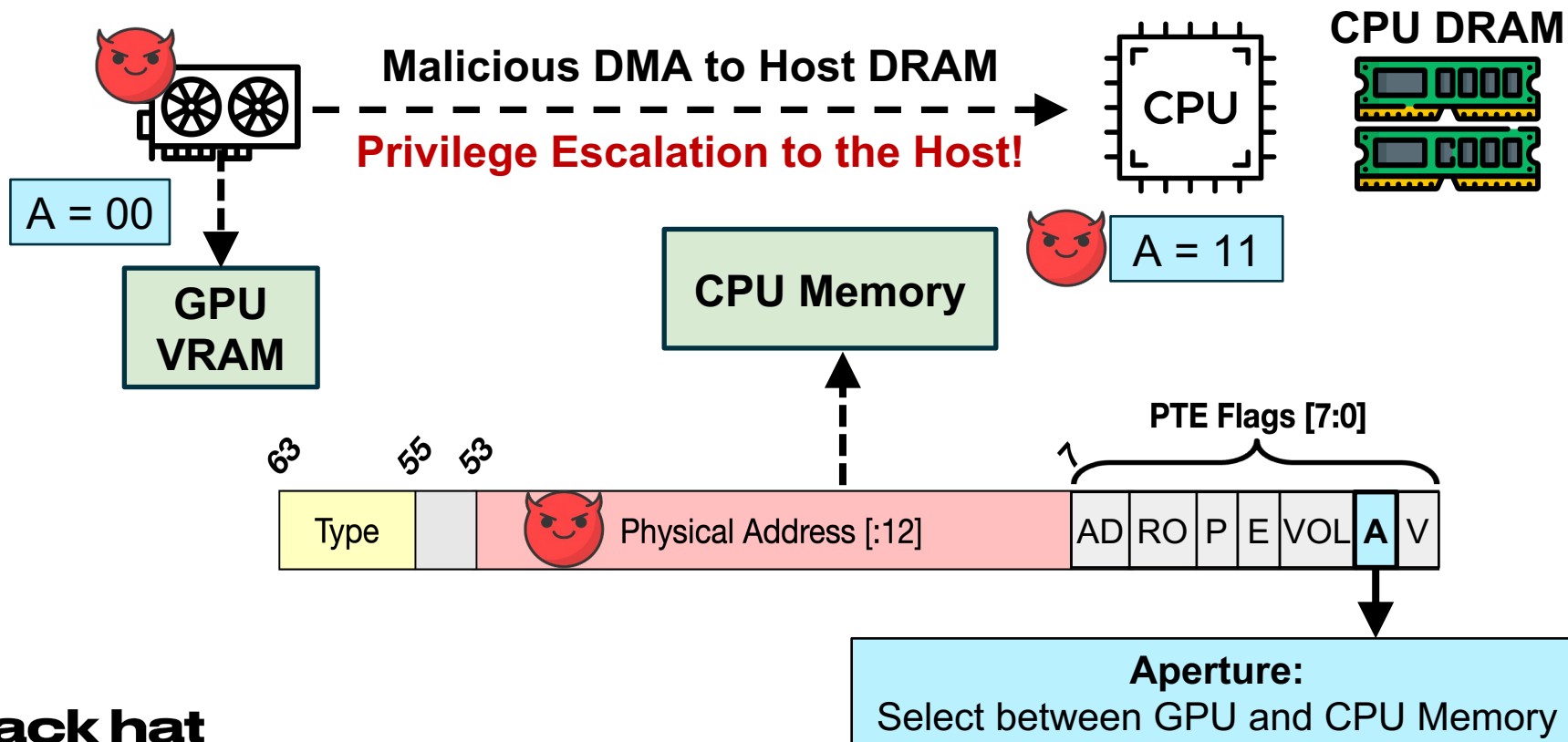
Victim

Leak **ML model weights** and **secret keys** from GPU memory

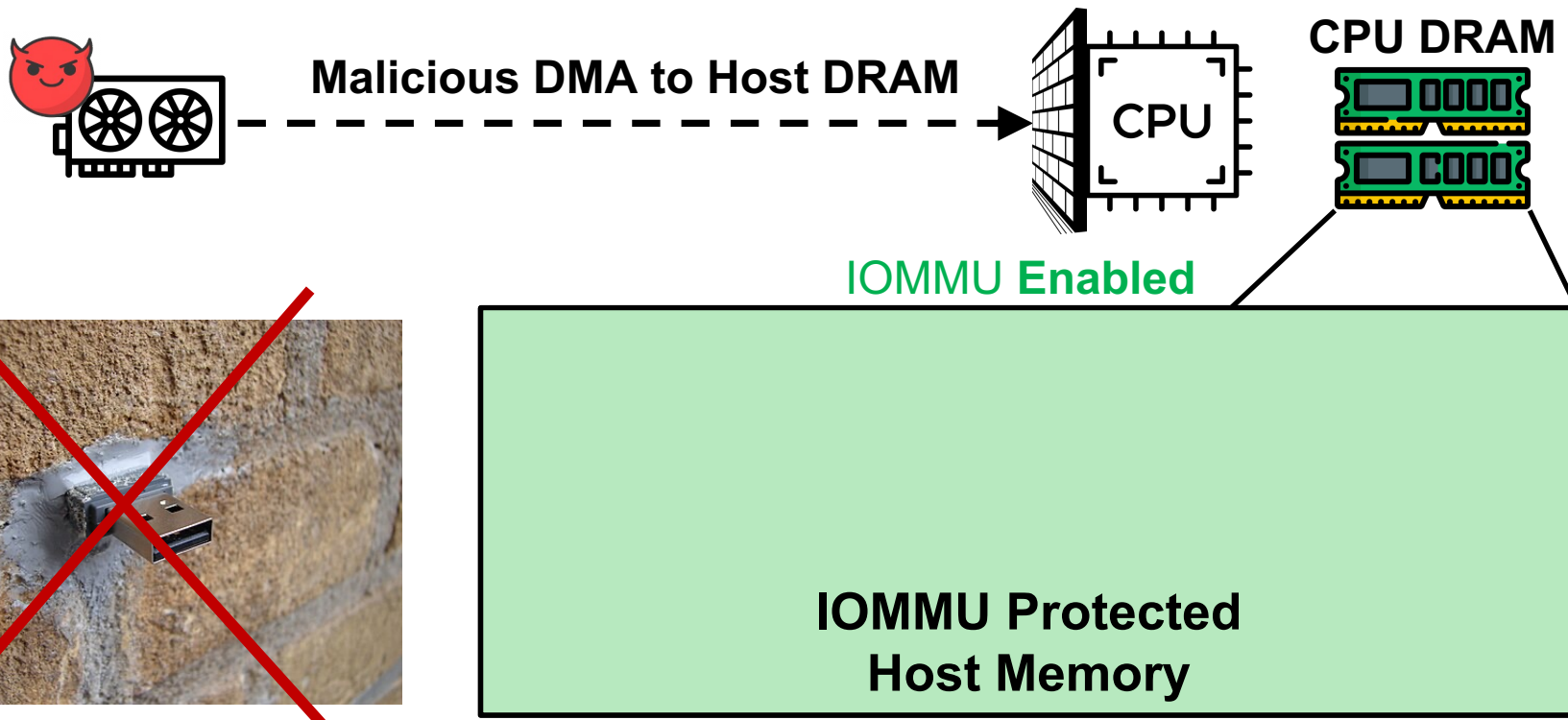
Stealthily tamper cuBLAS code & universally degrade ML accuracy

Let's do even better!

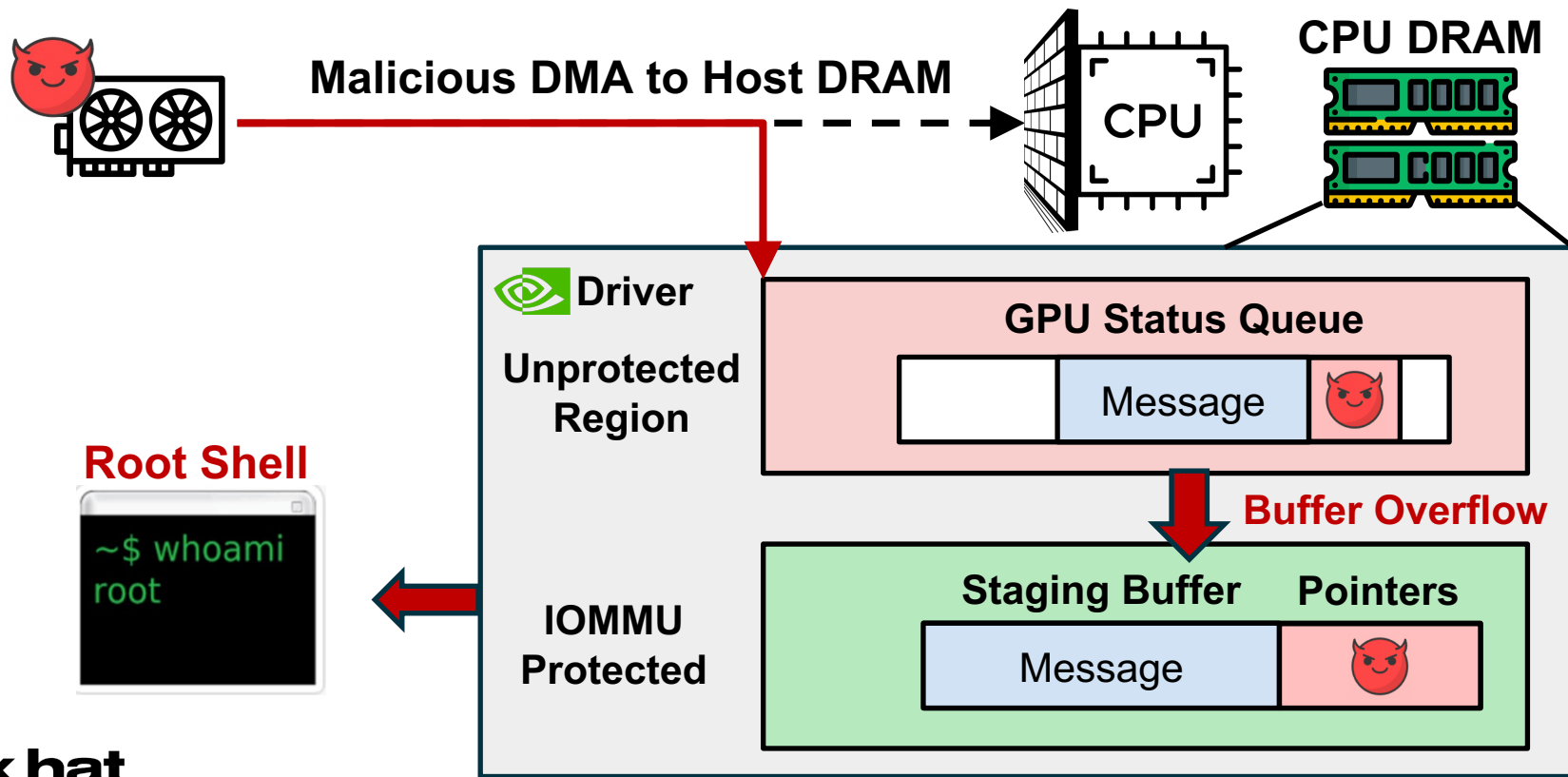
Exploit 3: CPU-side Privilege Escalation



Exploit 3: CPU-side Privilege Escalation

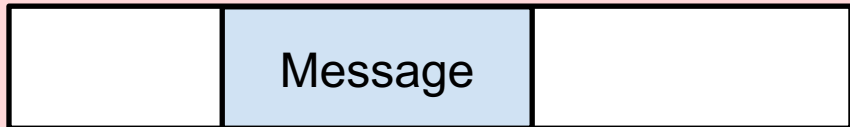


Exploit 3: CPU-side Privilege Escalation



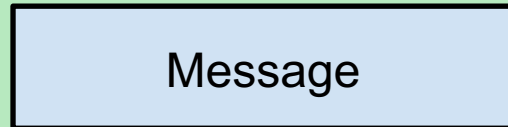
Exploit 3: CPU-side Privilege Escalation

GPU Status Queue



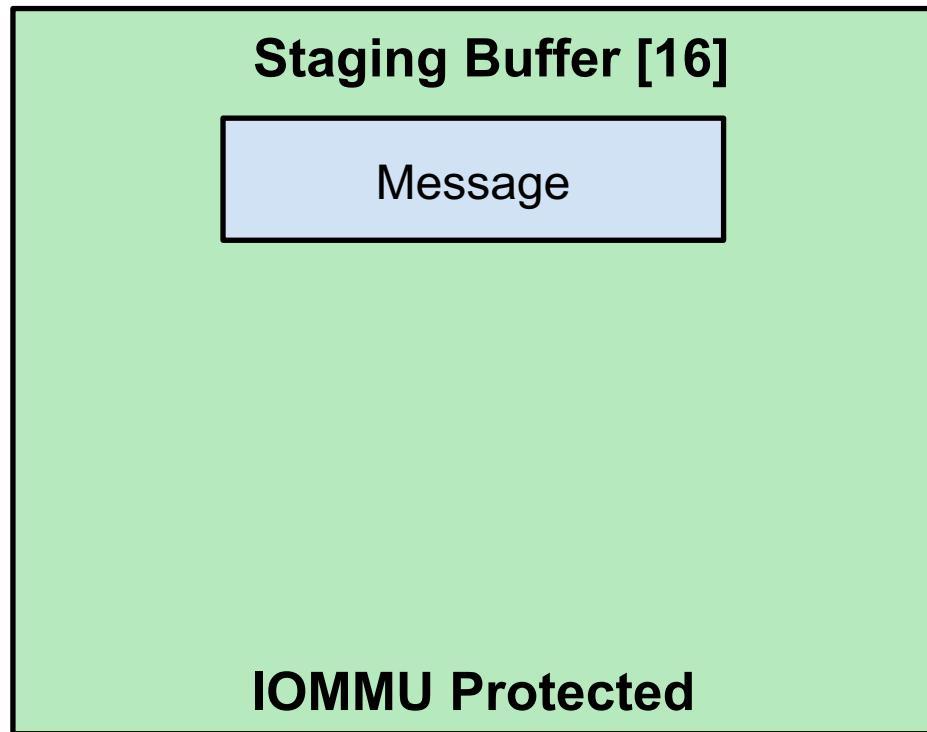
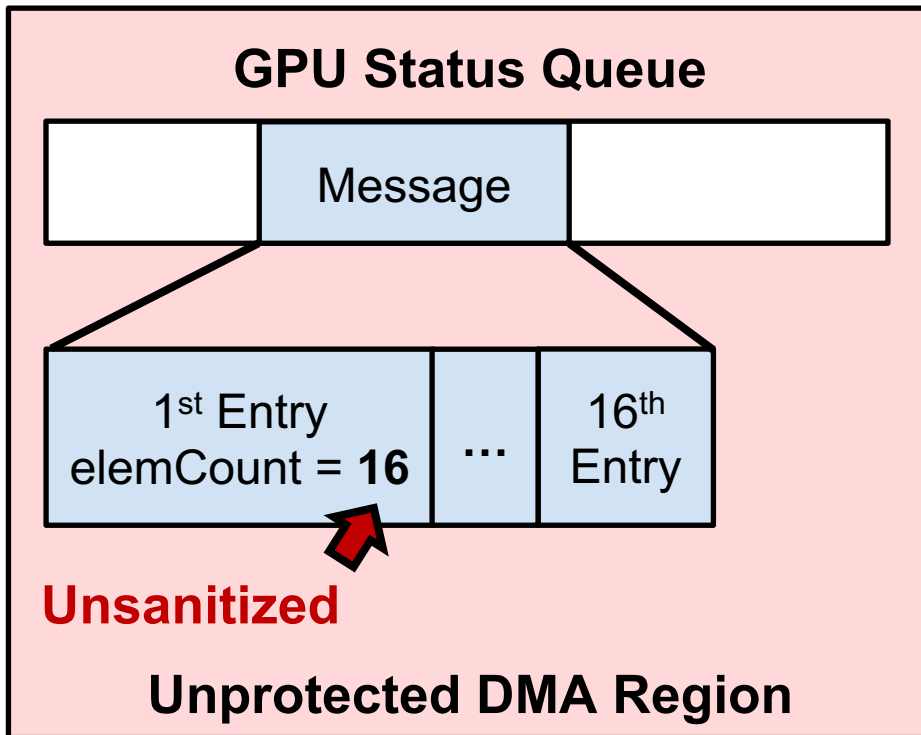
Unprotected DMA Region

Staging Buffer

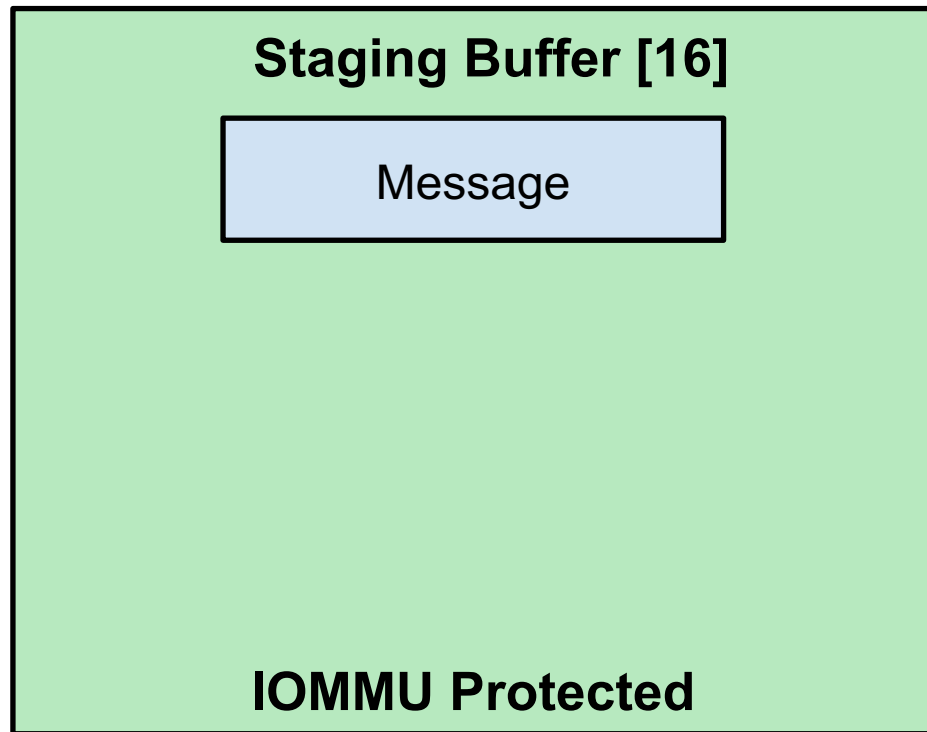
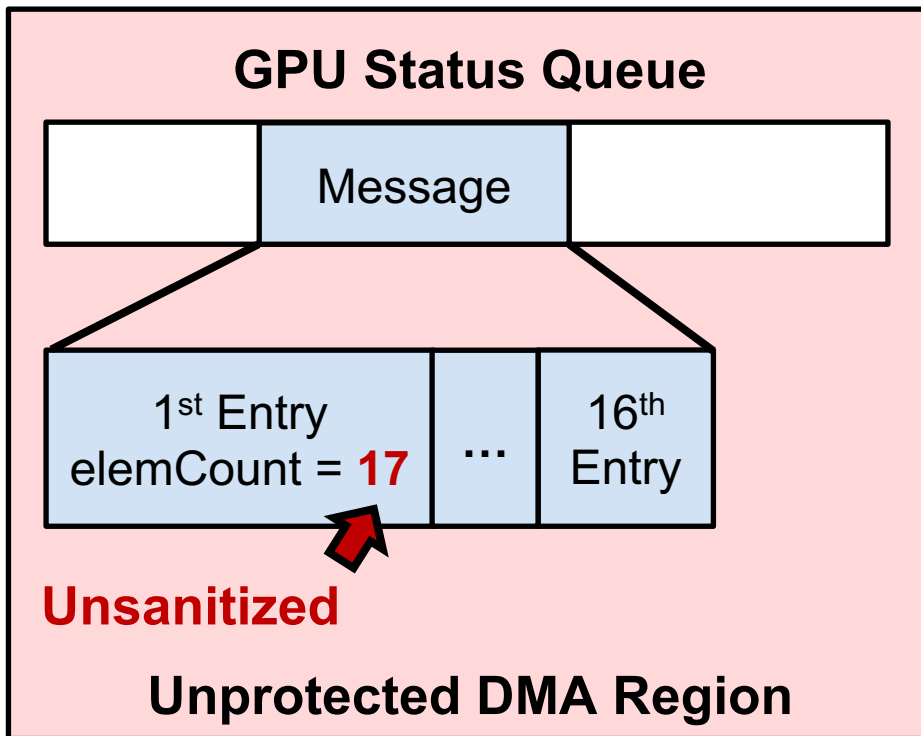


IOMMU Protected

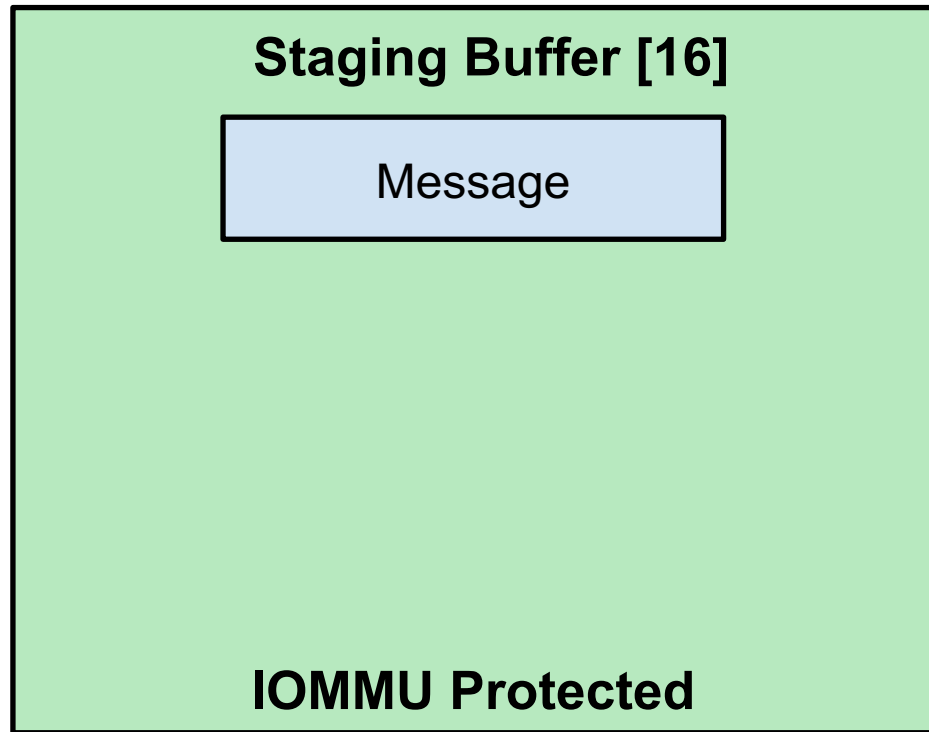
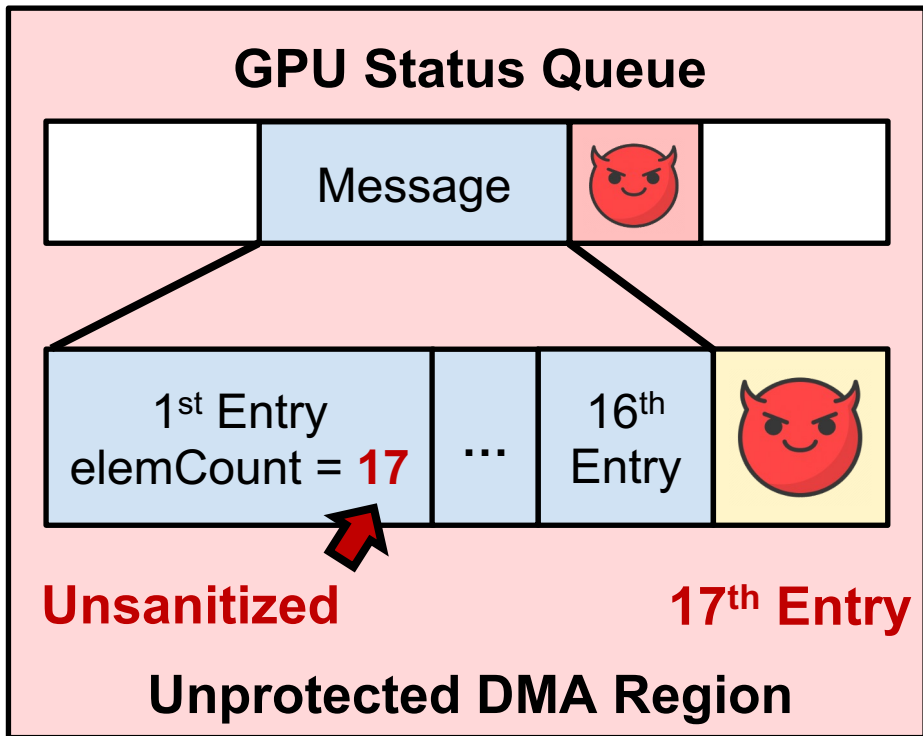
Exploit 3: CPU-side Privilege Escalation



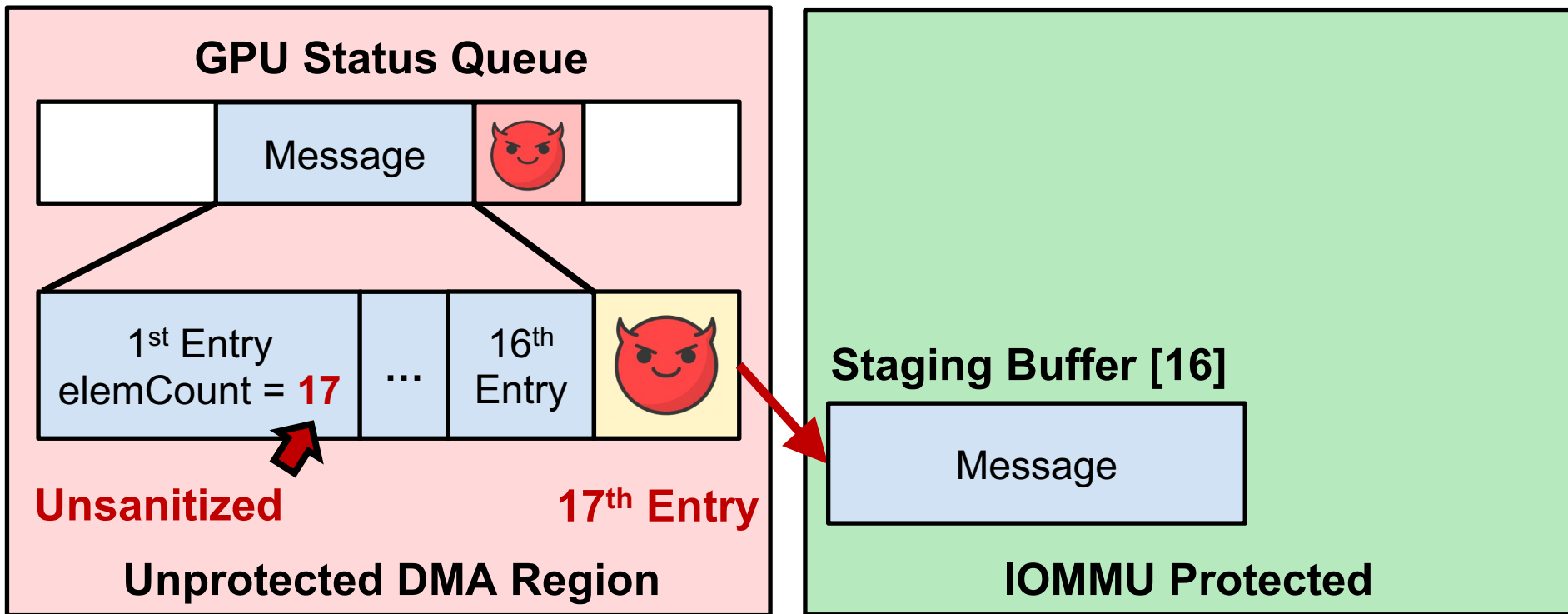
Exploit 3: CPU-side Privilege Escalation



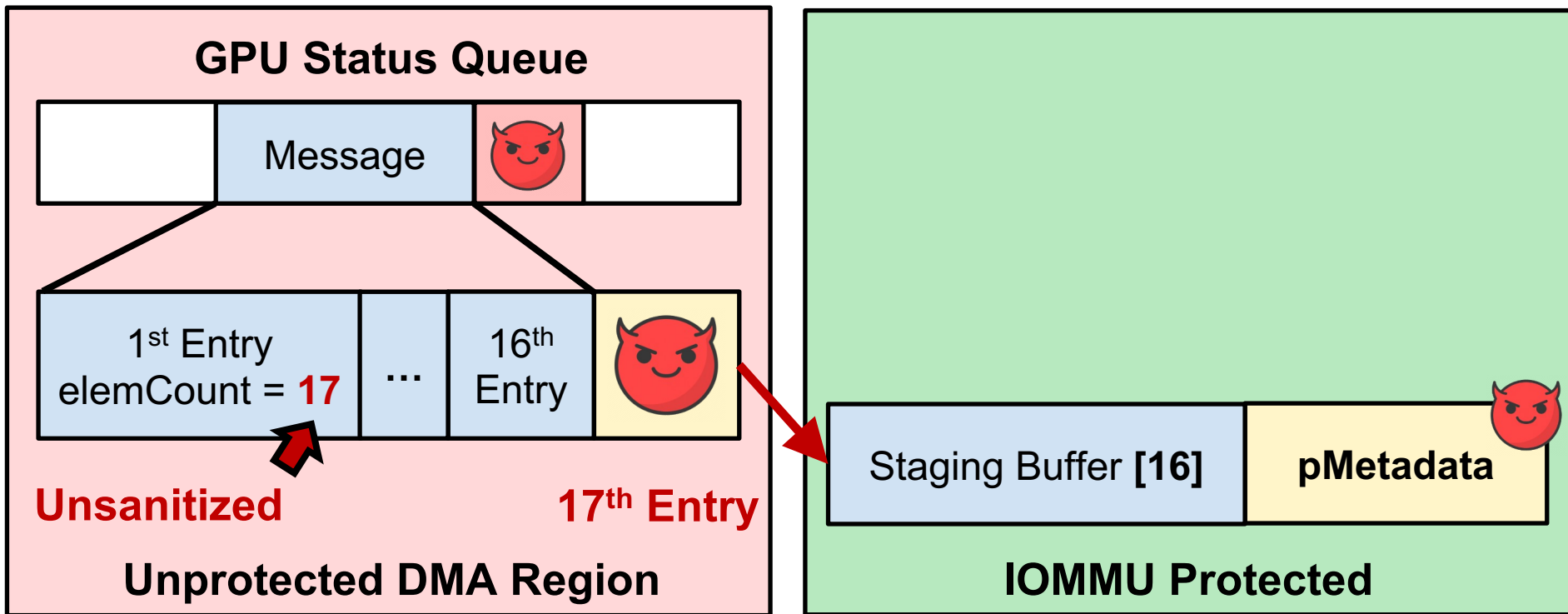
Exploit 3: CPU-side Privilege Escalation



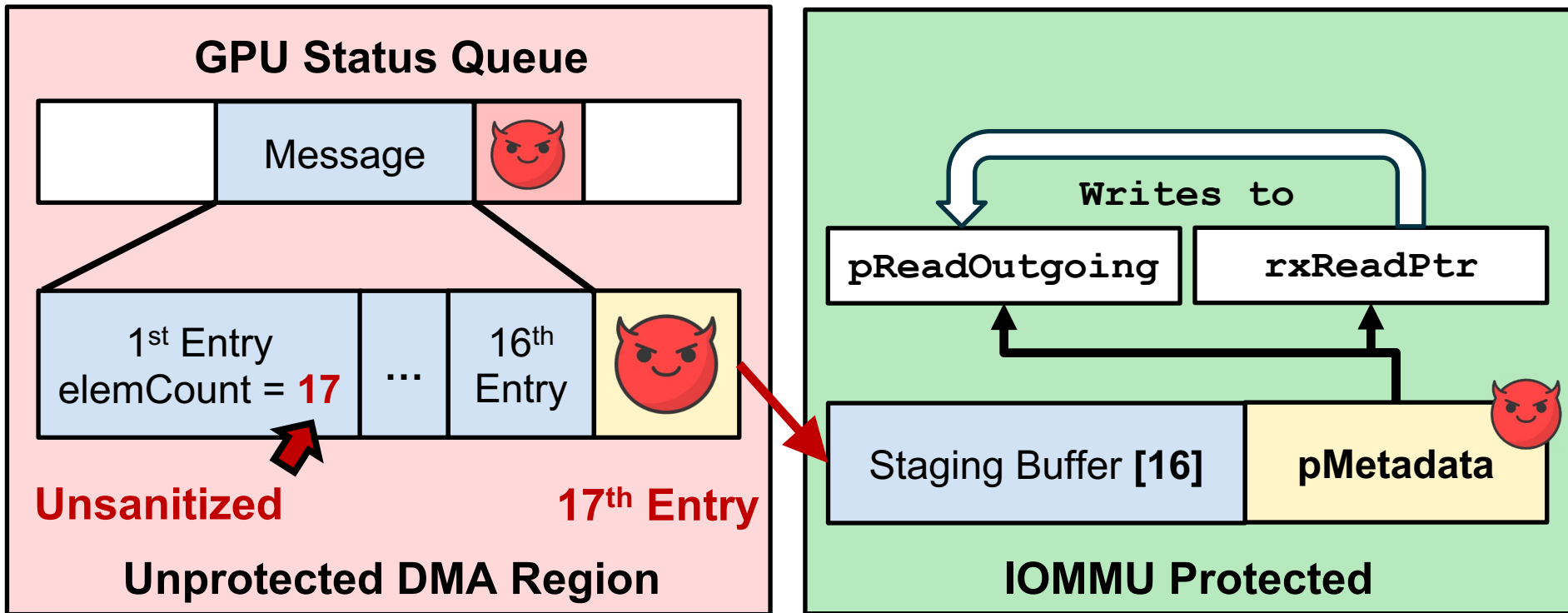
Exploit 3: CPU-side Privilege Escalation



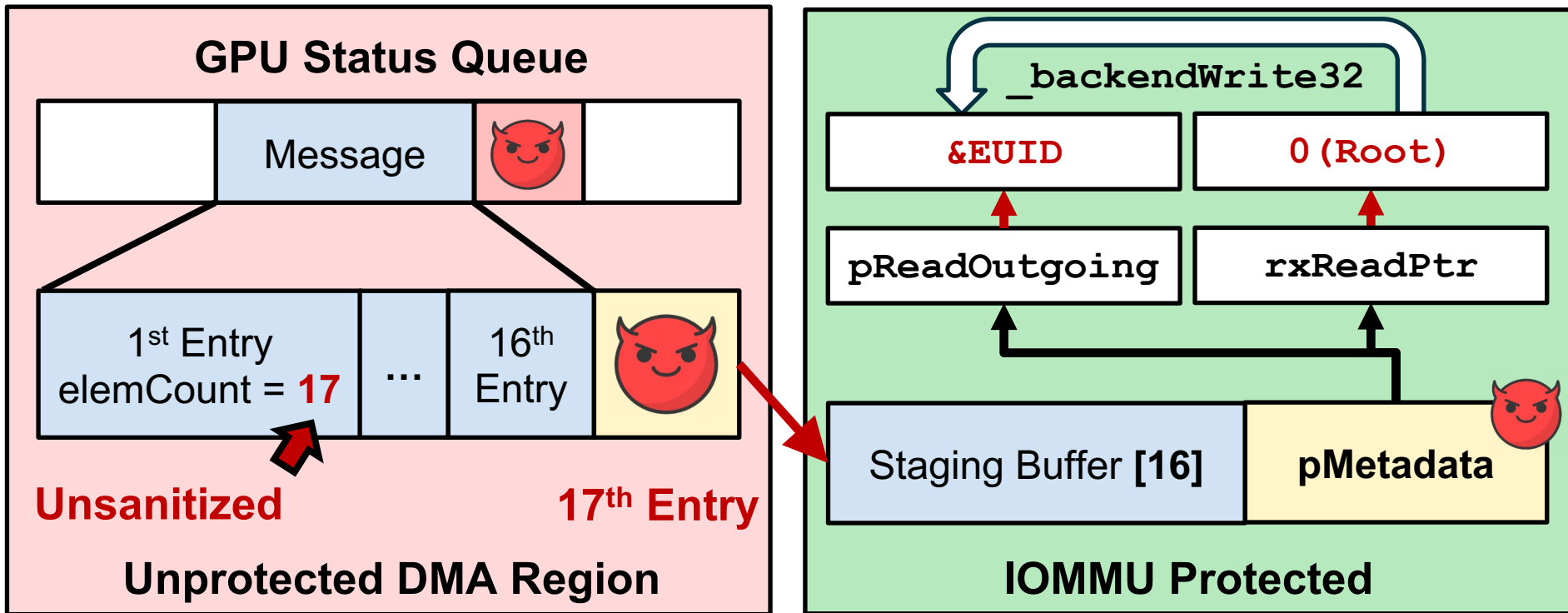
Exploit 3: CPU-side Privilege Escalation



Exploit 3: CPU-side Privilege Escalation



Exploit 3: CPU-side Privilege Escalation



Takeaways

GPU Rowhammer is a real threat!

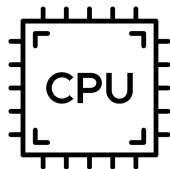
- We went from random corruption to targeted attacks that take over GPUs.

CPU-side Driver is not safe from a malicious GPU.

- GPUBreach extended the exploit to the host, gaining powerful primitives
- Driver's lack of sanitization on input from the GPU is the main culprit.

Rethinking GPU Security Assumptions

Takeaways

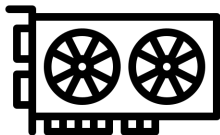


Sanitize GPU-Side Inputs in Driver

PT Region

DATA

Isolate GPU Page Tables from Data



Error Correction Codes in GPU DRAM



Principled RH Defense in GPU DRAM

Responsible Disclosure



Responsibly disclosed to
NVIDIA & Cloud Vendors

GPUBreach has been reported to respective parties.

- **Vendor:** NVIDIA. **CSPs:** Google, Microsoft, etc
- NVIDIA acknowledged the risks, while the CSPs are discussing mitigation approaches with their respective engineering teams.

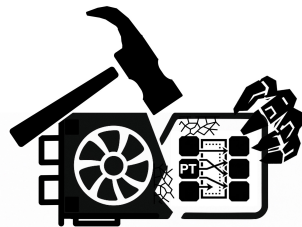


Awarded
Google Bug Bounty



Awarded IEEE S&P
Distinguished Paper

Thanks!



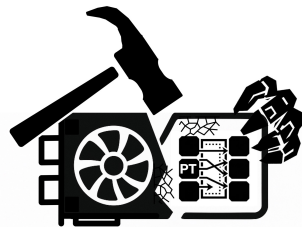
Collaborators:

Joyce Qu, Joseph Zhu, David Lie, and Gururaj Saileshwar

Presentation Coach:

Philip Young

Thanks!



Contacts:



www.gpubreach.ca

Chris:

email: shaopenglin@cs.toronto.edu

personal site: <https://shaopenglin.github.io>

Yuqin:

email: me@yqyan.com

Guozhen:

email: gzh.ding@mail.utoronto.ca

personal site : <https://www.guozhen.dev/>